Project Code Documentation: Mapping Legislative Attention to the Social Determinants of Mental Health in State Mental Health Bills

Jacob Federbusch

2026-04-20

Table of contents

1 Project Goal and CRISP-DM Roadmap **2**

2 Environment and Packages **3**

3 Paths and Data Inputs **4**

4 Data Import and Cleaning **5**
4.1 1) Build metadata from local LegiScan archive JSON (Python) 5
4.2 2) Filter to mental-health candidate bills (Python) 7
4.3 3) Retrieve and assemble full bill texts (Python) 8

5 Preprocessing and Feature Extraction **12**
5.1 4) Build corpus table (R) 12
5.2 5) Political metadata and join (R) 13
5.3 6) Domain dictionary (R) 14
5.4 7) Token and n-gram tables (R) 17
5.5 8) Named entities (R + spaCy) 21

6 Modeling and Analysis **23**
6.1 RQ1: Frequency and TF-IDF by domain terms 23
6.2 RQ2: Domain mention shares by state/year/party 24
6.3 RQ3: High-vs-low domain bills, distinct language and entities 25

7 RQ4: Latent Topics and Domain Linkages **34**
7.1 Topic table . . . 34
7.2 4.1 Load topic model and scored corpus . . . 37
7.3 4.2 RQ4a: Topic prevalence . . . 37
7.4 4.3 RQ4b: Topic–domain co-occurrence . . . 38
7.5 4.4 RQ4c: State and session-year variation . . . 40

8 Evaluation, Plots, and Output Checks **40**
8.1 Base Theme . . . 41
8.2 RQ1 Visualizations . . . 41
8.3 RQ2 Visualizations . . . 43
8.4 RQ3 Visualizations . . . 44
8.5 RQ4 Visualizations . . . 55

1 Project Goal and CRISP-DM Roadmap

This document is the executable workflow for the project. It shows how the corpus was built and analyzed, with short narrative and runnable code that recreates the tables and figures used in the final paper.

Research questions addressed in this workflow:

- **RQ1.** How frequently do state mental health bills mention specific social-determinant domains, and which domains are most prominent?
- **RQ2.** How does domain language vary by state, session year, and party control?
- **RQ3.** What terms, phrases, and named entities distinguish bills with high versus low levels of social-determinant language?
- **RQ4.** What latent topics emerge in the bill corpus, and how do they relate to domain scores and state/year variation?

CRISP-DM roadmap used here:

1. Business/Data understanding: define the corpus of state mental health bills and the social-determinant measurement logic.
2. Data preparation: retrieve bill text, clean and tokenize text, and build analysis-ready tables (corpus, dictionary, token, n-gram, entity, and topic tables).
3. Modeling/analysis: implement the RQ1–RQ4 analyses (domain term frequencies and TF-IDF, n-gram patterns, named entities, and LDA topics) using those tables.
4. Evaluation: run diagnostics, generate the RQ-specific summary tables, and save the figures referenced in the project paper.

2 Environment and Packages

This chunk loads the R–Python bridge and points reticulate to the textmining Conda environment so that Python tools used in the project are available inside this Quarto document.

```
library(reticulate)
use_condaenv("textmining", required = TRUE)
```

The next chunk loads the core R packages used across data preparation, modeling, and visualization.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.2     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

```
library(readr)
library(readtext)
library(tidytext)
library(topicmodels)
library(textstem)
```

```
Loading required package: koRpus.lang.en
Loading required package: koRpus
Loading required package: sylly
For information on available language packages for 'koRpus', run

  available.koRpus.lang()

and see ?install.koRpus.lang()
```

```
Attaching package: 'koRpus'

The following object is masked from 'package:readr':

    tokenize
```

```r
library(spacyr)
library(forcats)
library(scales)
```

```
Attaching package: 'scales'

The following object is masked from 'package:purrr':

    discard

The following object is masked from 'package:readr':

    col_factor
```

```r
library(ggplot2)
library(ggrepel)
```

3 Paths and Data Inputs

This chunk ensures that the expected data, raw-text, processed-text, and figure directories exist before any files are read or written.

```r
# Ensure expected directories exist for data, raw text, processed text, and figures
dir.create("data", showWarnings = FALSE, recursive = TRUE)
dir.create("output/figures", showWarnings = FALSE, recursive = TRUE)
dir.create("raw_billtext", showWarnings = FALSE, recursive = TRUE)
dir.create("bill_txt", showWarnings = FALSE, recursive = TRUE)
```

4 Data Import and Cleaning

4.1 1) Build metadata from local LegiScan archive JSON (Python)

This chunk scans a local archive of LegiScan JSON bill records (downloaded using the documented LegiScan API and datasets) to build the metadata.csv table with core bill-, session-, and sponsor-level fields used throughout the pipeline.

```
# One-time metadata build from local LegiScan archives (already run).

import json
import csv
from pathlib import Path

bill_root = Path("unzipped_json/")
out_csv = Path("metadata.csv")

all_json = list(bill_root.rglob("*.json"))
bill_json_files = [f for f in all_json if f.parent.name == "bill"]
print(f"Found {len(bill_json_files)} bill JSON files")

rows = []

def clean_date(d):
    if not d or d == "0000-00-00":
        return None
    return d

for json_file in bill_json_files:
    try:
        data = json.loads(json_file.read_text(encoding="utf-8"))
        bill = data.get("bill", {})
        session = bill.get("session", {})
        texts = bill.get("texts", [])
        sponsors = bill.get("sponsors", [])

        introduced = None
        for t in texts:
            if str(t.get("type", "")).lower() == "introduced":
                introduced = t
                break
```

```
        if not introduced:
            continue

        primary_sponsor = sponsors[0] if sponsors else {}
        body_code = bill.get("body", "")
        chamber_map = {"A": "House", "H": "House", "S": "Senate", "J": "Joint", "": "Unknown"

        rows.append({
            "json_file": str(json_file),
            "bill_id": bill.get("bill_id"),
            "state": bill.get("state"),
            "bill_number": bill.get("bill_number"),
            "bill_type": bill.get("bill_type"),
            "session_id": bill.get("session_id"),
            "session_year": session.get("year_start"),
            "session_title": session.get("session_title"),
            "special_session": session.get("special", 0),
            "body_code": body_code,
            "chamber": chamber_map.get(body_code, body_code),
            "title": bill.get("title"),
            "description": bill.get("description"),
            "status_date": clean_date(bill.get("status_date")),
            "doc_id": introduced.get("doc_id"),
            "date_introduced": clean_date(introduced.get("date")),
            "text_type": introduced.get("type"),
            "mime": introduced.get("mime"),
            "source_url": introduced.get("url"),
            "state_link": introduced.get("state_link"),
            "sponsor_name": primary_sponsor.get("name"),
            "sponsor_party": primary_sponsor.get("party"),
            "sponsor_role": primary_sponsor.get("role"),
        })
    except Exception as e:
        print(f"Skipping {json_file}: {e}")

print(f"Rows collected: {len(rows)}")

if rows:
    with out_csv.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=rows[0].keys())
        writer.writeheader()
        writer.writerows(rows)
```

```
    print(f"Wrote {len(rows)} rows to {out_csv}")
```

4.2 2) Filter to mental-health candidate bills (Python)

Use title/description keywords to create the screened corpus input table.

```
# One-time filtering to mental-health bills from full metadata (already run).

import pandas as pd

df = pd.read_csv("metadata.csv")
print(f"Total bills: {len(df)}")

keywords = [
    "mental health", "mental illness", "mental disorder",
    "behavioral health", "psychiatric", "psychiatric disorder",
    "psycholog", "psychosis", "psychotic", "schizophrenia", "bipolar",
    "suicide", "suicidal", "crisis intervention",
    "civil commitment", "involuntary commitment",
    "post-traumatic", "trauma-informed",
    "substance use", "substance abuse", "substance use disorder",
    "co-occurring disorder", "addiction",
    "community mental health", "inpatient psychiatric",
    "outpatient psychiatric", "inpatient mental health",
    "mental health court", "peer support specialist",
    "serious emotional disturbance", "eating disorder",
    "depression", "anxiety disorder",
]

pattern = "|".join(keywords)

filtered = df[
    df["title"].str.contains(pattern, case=False, na=False) |
    df["description"].str.contains(pattern, case=False, na=False)
]

print(f"Mental-health bills: {len(filtered)}")
filtered["state"].value_counts()

filtered.to_csv("metadata_mental_health.csv", index=False)
```

4.3 3) Retrieve and assemble full bill texts (Python)

This chunk retrieves the full text of each screened bill from LegiScan's getBillText API endpoint using the doc_id stored in metadata_mental_health.csv. Because the endpoint returns a Base64-encoded doc field and bill texts arrive with different MIME types, the script decodes the payload, saves raw API responses for reproducibility, writes HTML or plain-text bills directly to bill_txt/, and saves PDF bills to raw_billtext/ for separate extraction in the next step. Using separate raw and processed directories preserves the original source material while creating standardized UTF-8 plain-text files for downstream analysis.

```
import requests
import base64
import json
import time
import pandas as pd
import os
from pathlib import Path
from bs4 import BeautifulSoup
import re

API_KEY = os.environ.get("LEGISCAN_KEY", "YOUR_API_KEY_HERE")
if API_KEY == "YOUR_API_KEY_HERE":
    raise ValueError("Set LEGISCAN_KEY in your environment before running this chunk.")

df = pd.read_csv("metadata_mental_health.csv")

raw_dir = Path("raw_billtext")
txt_dir = Path("bill_txt")
raw_dir.mkdir(exist_ok=True)
txt_dir.mkdir(exist_ok=True)

failed = []
skipped = 0
retrieved = 0

for i, row in df.iterrows():
    doc_id = int(row["doc_id"])
    state = row["state"]
    bill_number = str(row["bill_number"]).replace("/", "-")
    mime = str(row.get("mime", ""))

    txt_file = txt_dir / f"{state}_{bill_number}_{doc_id}.txt"
    raw_file = raw_dir / f"{state}_{bill_number}_{doc_id}.json"
```

```python
    # Skip if already retrieved
    if txt_file.exists():
        skipped += 1
        continue

    try:
        url = f"https://api.legiscan.com/?key={API_KEY}&op=getBillText&id={doc_id}"
        r = requests.get(url, timeout=30)
        data = r.json()

        if data.get("status") != "OK":
            print(f"[{i}] FAILED {state} {bill_number}: status={data.get('status')}")
            failed.append({"bill_id": row["bill_id"], "doc_id": doc_id,
                           "state": state, "bill_number": bill_number,
                           "reason": data.get("status")})
            time.sleep(1)
            continue

        # Save raw JSON response for reproducibility
        raw_file.write_text(json.dumps(data, ensure_ascii=False), encoding="utf-8")

        # Decode Base64 doc field
        doc_b64 = data["text"].get("doc", "")
        if not doc_b64:
            failed.append({"bill_id": row["bill_id"], "doc_id": doc_id,
                           "state": state, "bill_number": bill_number,
                           "reason": "empty doc"})
            continue

        doc_bytes = base64.b64decode(doc_b64)

        # Branch by MIME type
        if "html" in mime.lower():
            html = doc_bytes.decode("utf-8", errors="replace")
            soup = BeautifulSoup(html, "html.parser")
            text = soup.get_text(separator=" ", strip=True)
            text = re.sub(r"\s+", " ", text).strip()
            txt_file.write_text(text, encoding="utf-8")

        elif "pdf" in mime.lower():
            pdf_file = raw_dir / f"{state}_{bill_number}_{doc_id}.pdf"
            pdf_file.write_bytes(doc_bytes)
```

```
            failed.append({"bill_id": row["bill_id"], "doc_id": doc_id,
                           "state": state, "bill_number": bill_number,
                           "reason": "pdf_needs_extraction"})

        else:
            text = doc_bytes.decode("utf-8", errors="replace")
            text = re.sub(r"\s+", " ", text).strip()
            txt_file.write_text(text, encoding="utf-8")

        retrieved += 1
        if retrieved % 50 == 0:
            print(f"Retrieved: {retrieved}")

        time.sleep(0.5)

    except Exception as e:
        print(f"[{i}] ERROR {state} {bill_number}: {e}")
        failed.append({"bill_id": row["bill_id"], "doc_id": doc_id,
                       "state": state, "bill_number": bill_number,
                       "reason": str(e)})
        time.sleep(2)

print(f"\nDone. Retrieved: {retrieved} | Skipped: {skipped} | Issues: {len(failed)}")

if failed:
    pd.DataFrame(failed).to_csv("retrieval_issues.csv", index=False)
    print("Issues saved to retrieval_issues.csv")
```

This next chunk processes the backlog of PDF bills saved in raw_billtext/, converts them to UTF-8 plain text with pdfminer.six, and writes them into bill_txt/ so that every bill in the screened corpus has a standardized plain-text file for analysis.

```
from pdfminer.high_level import extract_text
from pathlib import Path
import re
import pandas as pd

raw_dir = Path("raw_billtext")
txt_dir = Path("bill_txt")
txt_dir.mkdir(exist_ok=True)
```

```python
pdf_files = list(raw_dir.glob("*.pdf"))
print(f"PDFs to process: {len(pdf_files)}")
```

```
PDFs to process: 789
```

```python
failed_pdfs = []
extracted = 0

for pdf_file in pdf_files:
    txt_file = txt_dir / (pdf_file.stem + ".txt")

    # Skip if already extracted
    if txt_file.exists():
        continue

    try:
        text = extract_text(str(pdf_file))

        # Discard near-empty extractions
        if not text or len(text.strip()) < 50:
            failed_pdfs.append({"file": pdf_file.name, "reason": "empty or too short"})
            continue

        text = re.sub(r"\s+", " ", text).strip()
        txt_file.write_text(text, encoding="utf-8")
        extracted += 1

        if extracted % 50 == 0:
            print(f"Extracted: {extracted}")

    except Exception as e:
        print(f"Failed: {pdf_file.name} - {e}")
        failed_pdfs.append({"file": pdf_file.name, "reason": str(e)})

print(f"\nDone. Extracted: {extracted} | Failed: {len(failed_pdfs)}")
```

```
Done. Extracted: 0 | Failed: 0
```

```
if failed_pdfs:
    pd.DataFrame(failed_pdfs).to_csv("pdf_extraction_failures.csv", index=False)
    print("Failures saved to pdf_extraction_failures.csv")
```

5 Preprocessing and Feature Extraction

5.1 4) Build corpus table (R)

This chunk constructs the main bill-level corpus by joining the plain-text bill files in bill_-txt/ to the screened LegiScan metadata in metadata_mental_health.csv. To keep session years comparable across states, it recodes Florida's 2022 and 2024 annual sessions to 2021 and 2023 so that all states are indexed by the start year of the regular session. The code then restricts the corpus to regular-session bills only (bill type B, special sessions excluded, introduced text version) and saves the resulting table as data/corpus_table_base.*, where each row represents one introduced mental health bill with its UTF-8 plain text and core metadata used downstream.

```
bill_texts <- readtext("bill_txt/*.txt", encoding = "UTF-8") |>
  mutate(
    doc_id_key = as.integer(str_extract(doc_id, "(?<=_)\\d+(?=\\.txt$)"))
  )

metadata <- read_csv("metadata_mental_health.csv", show_col_types = FALSE)

corpus_table <- metadata |>
  mutate(
    session_year = case_when(
      session_year == 2022 ~ 2021L,
      session_year == 2024 ~ 2023L,
      TRUE ~ as.integer(session_year)
    )
  ) |>
  filter(
    bill_type == "B",
    special_session == 0,
    text_type == "Introduced"
  ) |>
  left_join(bill_texts, by = c("doc_id" = "doc_id_key")) |>
  rename(
    bill_text = text,
```

```r
    bill_title = title,
    bill_version_used = text_type
  ) |>
  select(
    bill_id, state, session_year, session_title,
    bill_title, bill_text, date_introduced,
    chamber, bill_version_used, sponsor_party
  )

write_csv(corpus_table, "data/corpus_table_base.csv")
saveRDS(corpus_table, "data/corpus_table_base.rds")

corpus_table |>
  summarise(n_bills = n(), missing_text = sum(is.na(bill_text) | bill_text == ""))
```

```
# A tibble: 1 x 2
  n_bills missing_text
    <int>        <int>
1    3169            0
```

5.2 5) Political metadata and join (R)

This chunk adds political context by hand-coding party control for each state-session combination and merging it into the bill-level corpus. Party control is coded at the opening of each regular session and summarized in a three-category party_control_type variable (Democratic trifecta, Republican trifecta, or divided government), where a trifecta indicates unified party control of the governorship and both chambers. The resulting corpus_table_with_party_control.* files are used for descriptive subgroup comparisons by party control in later analyses, rather than for any causal claims.

```r
corpus_table <- read_csv("data/corpus_table_base.csv", show_col_types = FALSE)

party_control <- tribble(
  ~state, ~session_year, ~governor_party, ~upper_party, ~lower_party,
  "CA", 2021, "D", "D", "D", "CA", 2023, "D", "D", "D",
  "FL", 2021, "R", "R", "R", "FL", 2023, "R", "R", "R",
  "GA", 2021, "R", "R", "R", "GA", 2023, "R", "R", "R",
  "IL", 2021, "D", "D", "D", "IL", 2023, "D", "D", "D",
  "NY", 2021, "D", "D", "D", "NY", 2023, "D", "D", "D",
  "OH", 2021, "R", "R", "R", "OH", 2023, "R", "R", "R",
  "PA", 2021, "D", "R", "R", "PA", 2023, "D", "R", "D",
```

```r
  "TX", 2021, "R", "R", "R", "TX", 2023, "R", "R", "R",
  "WA", 2021, "D", "D", "D", "WA", 2023, "D", "D", "D",
  "WI", 2021, "D", "R", "R", "WI", 2023, "D", "R", "R"
) |>
  mutate(
    party_control_type = case_when(
      governor_party == "D" & upper_party == "D" & lower_party == "D" ~ "Democratic",
      governor_party == "R" & upper_party == "R" & lower_party == "R" ~ "Republican",
      TRUE ~ "Divided"
    )
  )

corpus_table <- corpus_table |>
  left_join(party_control, by = c("state", "session_year"))

write_csv(party_control, "data/party_control.csv")
write_csv(corpus_table, "data/corpus_table_with_party_control.csv")
saveRDS(corpus_table, "data/corpus_table_with_party_control.rds")

corpus_table |>
  count(party_control_type)
```

```
# A tibble: 3 x 2
  party_control_type     n
  <chr>              <int>
1 Democratic          2139
2 Divided              251
3 Republican           779
```

5.3 6) Domain dictionary (R)

This chunk defines the final domain dictionary used to score bills on the six social-determinant domains. The dictionary was developed deductively from the SDOH literature and Healthy People 2030, with an inductive check on a 30-bill sample to add high-signal terms and remove ambiguous entries. The resulting domain_dictionary table is saved to data/domain_dictionary.csv for reuse in downstream analyses.

```r
domain_dictionary <- tribble(
  ~domain,              ~term_or_phrase,             ~term_type, ~notes,
  # domain          = one of the 6 SDOH domain labels (short, consistent)
  # term_or_phrase  = the actual word or phrase to search for (lowercase)
```

```
# term_type       = "unigram", "bigram", or "trigram"
# notes           = brief justification or flag

"economic_stability",  "poverty",                    "unigram",  "core term",
"economic_stability",  "unemployment",               "unigram",  "core term",
"economic_stability",  "food insecurity",            "bigram",   "core term",
"economic_stability",  "financial assistance",       "bigram",   "core term",
"economic_stability",  "employment",                 "unigram",  "source: HP2030",
"economic_stability",  "food security",              "bigram",   "source: HP2030",
"economic_stability",  "housing cost",               "bigram",   "source: HP2030",
"economic_stability",  "housing instability",        "bigram",   "source: paper",
"economic_stability",  "housing loss",               "bigram",   "source: paper",
"economic_stability",  "income",                     "unigram",  "source: paper; HP2030",
"economic_stability",  "job insecurity",             "bigram",   "source: paper",
"economic_stability",  "marital instability",        "bigram",   "source: paper",
"economic_stability",  "socioeconomic disadvantage","bigram",   "source: paper",
"economic_stability",  "child welfare",              "bigram",   "source: sample bills",
"economic_stability",  "safety net",                 "bigram",   "Explicit reference to stru
"economic_stability",  "human services",             "bigram",   "Social safety net infrasti

"education_access",    "early childhood",            "bigram",   "source: HP2030",
"education_access",    "graduation",                 "unigram",  "source: HP2030",
"education_access",    "job training",               "bigram",   "source: HP2030",
"education_access",    "workforce training",         "bigram",   "source: sample bills",
"education_access",    "literacy",                   "unigram",  "source: paper; HP2030",
"education_access",    "education",                  "unigram",  "Structural social conditio
"education_access",    "child care",                 "bigram",   "Early childhood structural
"education_access",    "day care",                   "bigram",   "Early childhood structural
"education_access",    "scholarship program",        "bigram",   "Educational access interve

"healthcare_access",   "medicaid",                   "unigram",  "Public insurance program;
"healthcare_access",   "health care access",         "trigram",  "source: paper",
"healthcare_access",   "health care delivery",       "trigram",  "source: paper",
"healthcare_access",   "health systems",             "bigram",   "source: paper",
"healthcare_access",   "telehealth",                 "unigram",  "source: HP2030",
"healthcare_access",   "uninsured",                  "unigram",  "source: HP2030",
"healthcare_access",   "hospitals",                  "unigram",  "Structural care delivery i
"healthcare_access",   "crisis response",            "bigram",   "source: sample bills",
"healthcare_access",   "loan repayment",             "bigram",   "Provider shortage/workforc
"healthcare_access",   "managed care",               "bigram",   "Structural mechanism for l
"healthcare_access",   "health care coverage",       "trigram",  "source: sample bills",
"healthcare_access",   "developmental disabilities","bigram",   "source: n-gram diagnostics
```

```
"healthcare_access",   "designated crisis responder","trigram", "Structural crisis interver
"healthcare_access",   "behavioral health services","trigram",  "Extends behavioral health
"healthcare_access",   "emergency medical services","trigram",  "Structural emergency care
"healthcare_access",   "health care provider",      "trigram",  "Workforce access signal; r
"healthcare_access",   "health care professional",  "trigram",  "Workforce access signal",

"neighborhood_environment", "air quality",          "bigram",   "source: paper",
"neighborhood_environment", "built environment",    "bigram",   "source: paper; HP2030",
"neighborhood_environment", "green space",          "bigram",   "source: HP2030",
"neighborhood_environment", "housing",              "unigram",  "source: paper; HP2030",
"neighborhood_environment", "neighborhood",         "unigram",  "source: paper",
"neighborhood_environment", "physical environments","bigram",   "source: paper",
"neighborhood_environment", "pollution",            "unigram",  "source: paper",
"neighborhood_environment", "transportation",       "unigram",  "source: HP2030",
"neighborhood_environment", "unsafe physical environments","trigram","source: paper",
"neighborhood_environment", "violence",             "unigram",  "source: paper; HP2030",

"social_community",    "community context",         "bigram",   "source: paper",
"social_community",    "incarceration",            "unigram",  "source: HP2030",
"social_community",    "isolation",                "unigram",  "source: HP2030",
"social_community",    "lived experiences",        "bigram",   "source: paper",
"social_community",    "social disadvantage",      "bigram",   "source: paper",
"social_community",    "social support",           "bigram",   "source: paper; HP2030",
"social_community",    "community well-being",     "bigram",   "source: paper",
"social_community",    "substance abuse",          "bigram",   "Social condition with struc
"social_community",    "substance use",            "bigram",   "As a social condition (subs
"social_community",    "drug court",               "bigram",   "source: sample bills",
"social_community",    "mental health court",      "trigram",  "source: sample bills",
"social_community",    "housing crisis",           "bigram",   "source: sample bills",
"social_community",    "peer support",             "bigram",   "source: sample bills",
"social_community",    "community safety",         "bigram",   "source: sample bills",
"social_community",    "suicide prevention",       "bigram",   "Explicitly upstream-orient
"social_community",    "violence prevention",      "bigram",   "source: sample bills",
"social_community",    "family services",          "bigram",   "Social support infrastructu
"social_community",    "community mental health",  "trigram",  "Place-based community care
"social_community",    "law enforcement agency",   "trigram",  "Extends law enforcement big
"social_community",    "law enforcement officer",  "trigram",  "Extends law enforcement big
"social_community",    "addiction services provider","trigram","Substance use service deliv

"discrimination_equity","discrimination",          "unigram",  "source: paper; HP2030",
"discrimination_equity","disparity",               "unigram",  "source: paper; HP2030",
"discrimination_equity","equity",                  "unigram",  "source: paper; HP2030",
```

```r
  "discrimination_equity","health disparities",      "bigram",   "source: paper",
  "discrimination_equity","health equity",           "bigram",   "source: paper",
  "discrimination_equity","health inequities",       "bigram",   "source: paper",
  "discrimination_equity","human rights",            "bigram",   "source: paper",
  "discrimination_equity","inequities",              "unigram",  "source: paper",
  "discrimination_equity","marginalized populations","bigram",   "source: paper",
  "discrimination_equity","racism",                  "unigram",  "source: paper; HP2030",
  "discrimination_equity","social exclusion",        "bigram",   "source: paper",
  "discrimination_equity","stigma",                  "unigram",  "source: paper; HP2030",
  "discrimination_equity","structural inequities",   "bigram",   "source: paper",
  "discrimination_equity","structural racism",       "bigram",   "source: paper",
  "discrimination_equity","systemic barriers",       "bigram",   "source: paper",
  "discrimination_equity","systemic discrimination", "bigram",   "source: paper",
  "discrimination_equity","sickle cell",             "bigram",   "Signals racial equity fram
  "discrimination_equity","medical cannabis",        "bigram",   "Drug policy with documente
)

# Optional quick check of coverage by domain and term type
domain_dictionary |>
  count(domain, term_type) |>
  pivot_wider(names_from = term_type, values_from = n, values_fill = 0) |>
  mutate(total = rowSums(across(where(is.numeric))))
```

```
# A tibble: 6 x 5
  domain                    bigram unigram trigram total
  <chr>                      <int>   <int>   <int> <dbl>
1 discrimination_equity         12       6       0    18
2 economic_stability            12       4       0    16
3 education_access               6       3       0     9
4 healthcare_access              5       4       8    17
5 neighborhood_environment       4       5       1    10
6 social_community              14       2       5    21
```

```r
write_csv(domain_dictionary, "data/domain_dictionary.csv")
```

5.4 7) Token and n-gram tables (R)

These chunks construct the term-level features used throughout the analysis. The first chunk builds a token table and a lemmatized companion table after removing standard and custom legislative stopwords, and the second builds bigram and trigram tables, then attaches domain

labels to n-grams that match entries in the domain dictionary while filtering out markup artifacts.

The custom legal_stopwords list removes high-frequency procedural boilerplate and corpus constants so that downstream TF, TF-IDF, and dictionary measures reflect substantive policy language rather than drafting conventions. The token_table retains raw, unstemmed tokens for dictionary scoring, TF, and n-gram work, while token_table_lemma uses lemmatized tokens for the RQ1b TF-IDF step, where collapsing inflectional variants yields a more stable distinctiveness measure.

```
corpus_table <- read_csv("data/corpus_table_with_party_control.csv", show_col_types = FALSE)

data(stop_words)

legal_stopwords <- tibble(token = c(
  "shall", "section", "subsection", "pursuant", "thereof",
  "therein", "hereby", "herein", "notwithstanding", "provided",
  "effective", "act", "code", "chapter", "article",
  "subdivision", "paragraph", "clause", "amended", "enacted",
  "legislature", "assembly", "senate", "bill", "statute",

  # Procedural boilerplate from top-30 check
  "means", "including", "read", "revised", "period",
  "hundred", "provide", "division",

  # Corpus constants from top-30 check
  "health", "mental", "services", "department", "care",
  "treatment", "program", "public", "board", "medical",
  "information", "facility", "licensed", "license",
  "practice", "service",

  # Noise from second top-30 check
  "person", "persons", "date", "thousand", "include",
  "required", "requirements", "defined", "introduced", "rules",
  "subject", "time", "law",

  # State names
  "illinois", "california", "florida", "georgia", "new",
  "york", "ohio", "pennsylvania", "texas", "washington",
  "wisconsin",

  # Noise from third top-30 check
  "laws", "based", "authorized", "amount", "limited",
```

```r
  "provisions",

  # Month names
  "january", "february", "march", "april", "may", "june",
  "july", "august", "september", "october", "november", "december",

  # Noise from fourth top-30 check
  "accordance", "effect", "term",

  # Noise from fifth top-30 check
  "days", "purposes"
))

token_table <- corpus_table |>
  unnest_tokens(token, bill_text) |>
  anti_join(stop_words, by = c("token" = "word")) |>
  anti_join(legal_stopwords, by = "token") |>
  filter(str_length(token) > 2, !str_detect(token, "^\\d+$")) |>
  count(bill_id, token, name = "n", sort = FALSE)

token_table_lemma <- token_table |>
  mutate(lemma = lemmatize_words(token)) |>
  count(bill_id, lemma, wt = n, name = "n", sort = FALSE)

write_csv(token_table, "data/token_table.csv")
saveRDS(token_table, "data/token_table.rds")
write_csv(token_table_lemma, "data/token_table_lemma.csv")
saveRDS(legal_stopwords, "data/legal_stopwords.rds")

# quick token diagnostics
list(
  unique_tokens = n_distinct(token_table$token),
  mean_tokens_per_bill = token_table |> group_by(bill_id) |> summarise(t = sum(n), .groups =
)
```

```
$unique_tokens
[1] 34543

$mean_tokens_per_bill
[1] 884.113
```

```r
legal_stopwords <- readRDS("data/legal_stopwords.rds")
corpus_table <- read_csv("data/corpus_table_with_party_control.csv", show_col_types = FALSE)
domain_dictionary <- read_csv("data/domain_dictionary.csv", show_col_types = FALSE)

conflict_words <- c("health", "mental", "services", "care", "program", "medical", "law")
ngram_component_stopwords <- legal_stopwords |> filter(!token %in% conflict_words)

bigram_table <- corpus_table |>
  unnest_tokens(bigram, bill_text, token = "ngrams", n = 2) |>
  separate(bigram, into = c("word1", "word2"), sep = " ", remove = FALSE) |>
  filter(
    !word1 %in% stop_words$word, !word2 %in% stop_words$word,
    !word1 %in% ngram_component_stopwords$token, !word2 %in% ngram_component_stopwords$token
    !str_detect(word1, "^\\d+$"), !str_detect(word2, "^\\d+$"),
    str_length(word1) > 2, str_length(word2) > 2
  ) |>
  count(bill_id, bigram, name = "n", sort = FALSE)

trigram_table <- corpus_table |>
  unnest_tokens(trigram, bill_text, token = "ngrams", n = 3) |>
  separate(trigram, into = c("word1", "word2", "word3"), sep = " ", remove = FALSE) |>
  filter(
    !word1 %in% stop_words$word, !word2 %in% stop_words$word, !word3 %in% stop_words$word,
    !word1 %in% ngram_component_stopwords$token,
    !word2 %in% ngram_component_stopwords$token,
    !word3 %in% ngram_component_stopwords$token,
    !str_detect(word1, "^\\d+$"), !str_detect(word2, "^\\d+$"), !str_detect(word3, "^\\d+$")
  ) |>
  count(bill_id, trigram, name = "n", sort = FALSE)

dict_bigrams <- domain_dictionary |> filter(term_type == "bigram") |> select(term_or_phrase,
dict_trigrams <- domain_dictionary |> filter(term_type == "trigram") |> select(term_or_phrase

ngram_table <- bind_rows(
  bigram_table |> left_join(dict_bigrams, by = c("bigram" = "term_or_phrase")) |> rename(ngra
  trigram_table |> left_join(dict_trigrams, by = c("trigram" = "term_or_phrase")) |> rename(n
) |>
  select(bill_id, ngram_type, ngram, n, domain)

markup_ngrams <- tibble(ngram = c(
  # Bigram markup / header strings
  "coding words", "deletions words", "words stricken",
```

```
  "words underlined", "additions page", "source p.a",
  "assem bly", "explanation matter", "italics underscored",

  # Trigram markup / non-content strings
  "coding words stricken",      # markup phrase
  "deletions words underlined", # markup phrase
  "drug involved equals",        # bill amendment tracking string
  "mental hygiene law",         # New York Mental Hygiene Law citation is not a content signa
  "social services law",          # same issue as above; another NY statutory citation
  "title lrb103"             # clear non-content item found in plot
))

ngram_table <- ngram_table %>%
  anti_join(markup_ngrams, by = "ngram")

write_csv(ngram_table, "data/ngram_table.csv")
saveRDS(ngram_table, "data/ngram_table.rds")

ngram_table |>
  summarise(rows = n(), matched_domain_rows = sum(!is.na(domain)))
```

5.5 8) Named entities (R + spaCy)

This chunk runs named entity recognition on the full corpus using spaCy, consolidates entity spans, and builds an entity_table of ORG and GPE entities per bill. It then removes clearly non-organizational or overly generic strings (such as citation abbreviations, preamble phrases, and generic bodies like "senate") so that the remaining ORG and GPE entries correspond more closely to real institutions and places for RQ3 analyses.

```
corpus_table <- read_csv("data/corpus_table_with_party_control.csv", show_col_types = FALSE)

Sys.setenv(RETICULATE_PYTHON = "C:/Users/jakef/anaconda3/envs/textmining/python.exe")
spacy_initialize(model = "en_core_web_lg")
reticulate::py_run_string("nlp.max_length = 1600000")

MAX_CHARS <- 1500000
parsed_list <- list()

for (i in seq_len(nrow(corpus_table))) {
  bill_id <- corpus_table$bill_id[i]
  txt <- corpus_table$bill_text[i]
```

```r
  if (is.na(txt) || txt == "") next
  if (nchar(txt) > MAX_CHARS) txt <- substr(txt, 1, MAX_CHARS)

  out <- tryCatch({
    x <- txt
    names(x) <- paste0("bill_", bill_id)
    spacy_parse(x, entity = TRUE, pos = FALSE, tag = FALSE, lemma = FALSE)
  }, error = function(e) NULL)

  if (!is.null(out)) parsed_list[[length(parsed_list) + 1]] <- out
}

parsed <- bind_rows(parsed_list)
entities_raw <- entity_consolidate(parsed) |>
  filter(entity_type != "")

entity_table <- entities_raw |>
  mutate(
    bill_id = as.numeric(str_remove(doc_id, "bill_")),
    entity = str_squish(str_to_lower(token))
  ) |>
  select(bill_id, entity, entity_type) |>
  count(bill_id, entity, entity_type, name = "n", sort = FALSE) |>
  filter(entity_type %in% c("ORG", "GPE"))

org_stopwords <- tibble(entity = c(
  "sec",                                # section reference misclassified as ORG
  "rcw",                                # Revised Code of Washington citation
  "rlc",                                # legal citation abbreviation
  "hb", "sb",                           # bill number prefixes
  "state",                              # too generic
  "senate", "legislature",              # procedural bodies
  "the_general_assembly",               # procedural
  "commonwealth",                       # Pennsylvania's formal name - generic
  "the_people_of_the_state_of_illinois",  # preamble boilerplate
  "the_people_of_the_state_of_new_york",  # preamble boilerplate
  "department",                         # too generic - specific dept names stay
  "commission",                         # too generic - specific ones stay
  "board",                                     # too generic
  "blank",                                     # formatting text
  "general_assembly_state",                    # fragmented preamble text
  "general_assembly_state_of",                 # same
```

```
  "the_legislature_of_the_state_of_texas_:",  # preamble boilerplate with punctuation text
  "general_assembly",                         # procedural body
  "the_house_of_representatives",             # procedural body
  "lc",                                       # citation abbreviation
  "l.s.d.",                                   # citation abbreviation
  "california_legislature",   # procedural body - state-specific preamble
  "assembly",                 # procedural body
  "no_fiscal_committee",      # CA bill header string, not an organization
  "usc"                       # legal citation abbreviation (U.S. Code)
))

entity_table <- entity_table %>%
  anti_join(org_stopwords, by = "entity")

write_csv(entity_table, "data/entity_table.csv")
saveRDS(entity_table, "data/entity_table.rds")
spacy_finalize()

entity_table |>
  count(entity_type)
```

6 Modeling and Analysis

6.1 RQ1: Frequency and TF-IDF by domain terms

This chunk implements RQ1 by summarizing domain-specific terms and phrases at three levels. For RQ1a, it uses the raw token table and the unigram part of the domain dictionary to compute term frequency (TF) by domain, ranking dictionary-matched unigrams within each domain. For RQ1b, it joins the lemmatized token table to the same dictionary and computes bill-level TF-IDF, then averages TF-IDF within domains to identify distinctive lemmas. For RQ1c, it filters the n-gram table to domain-matched bigrams and trigrams and counts their occurrences by domain.

```
token_table <- read_csv("data/token_table.csv", show_col_types = FALSE)
ngram_table <- read_csv("data/ngram_table.csv", show_col_types = FALSE)
domain_dictionary <- read_csv("data/domain_dictionary.csv", show_col_types = FALSE)
token_table_lemma <- read_csv("data/token_table_lemma.csv", show_col_types = FALSE)

unigram_dict <- domain_dictionary |> filter(term_type == "unigram")
```

```r
rq1a_tf_terms <- token_table |>
  inner_join(unigram_dict, by = c("token" = "term_or_phrase")) |>
  count(domain, token, sort = TRUE, name = "tf") |>
  group_by(domain) |>
  mutate(rank = row_number()) |>
  ungroup()

rq1b_tfidf_terms <- token_table_lemma |>
  inner_join(unigram_dict, by = c("lemma" = "term_or_phrase")) |>
  count(bill_id, domain, lemma, wt = n, name = "tf") |>
  bind_tf_idf(term = lemma, document = bill_id, n = tf) |>
  group_by(domain, lemma) |>
  summarize(mean_tfidf = mean(tf_idf), .groups = "drop") |>
  group_by(domain) |>
  arrange(desc(mean_tfidf), .by_group = TRUE) |>
  mutate(rank = row_number()) |>
  ungroup()

rq1c_bigrams <- ngram_table |>
  filter(ngram_type == "bigram", !is.na(domain)) |>
  count(domain, ngram, sort = TRUE, name = "count")

rq1c_trigrams <- ngram_table |>
  filter(ngram_type == "trigram", !is.na(domain)) |>
  count(domain, ngram, sort = TRUE, name = "count")

write_csv(rq1a_tf_terms, "data/rq1a_tf_terms.csv")
write_csv(rq1b_tfidf_terms, "data/rq1b_tfidf_terms.csv")
write_csv(rq1c_bigrams, "data/rq1c_bigrams.csv")
write_csv(rq1c_trigrams, "data/rq1c_trigrams.csv")
```

6.2 RQ2: Domain mention shares by state/year/party

This chunk computes the share of bills that mention each social-determinant domain by state, session year, and party control. For each domain, it uses the domain dictionary to build a binary indicator showing whether a bill's text contains at least one matching term, then summarizes these indicators into proportions within each grouping. Session year is coded as the start year of the regular session (Florida's 2022 and 2024 annual sessions are aligned to 2021 and 2023 for comparability with biennial states), and party_control_type provides the Democratic/Republican/divided groupings used for descriptive comparisons.

```
corpus <- read_csv("data/corpus_table_with_party_control.csv", show_col_types = FALSE)
domain_dict <- read_csv("data/domain_dictionary.csv", show_col_types = FALSE)
domains <- unique(domain_dict$domain)

for (dom in domains) {
  terms <- domain_dict$term_or_phrase[domain_dict$domain == dom]
  pattern <- str_c(terms, collapse = "|")
  corpus[[paste0("domain_", dom)]] <- str_detect(corpus$bill_text, regex(pattern, ignore_case
}

domain_cols <- grep("^domain_", names(corpus), value = TRUE)
corpus_long <- corpus |>
  pivot_longer(cols = all_of(domain_cols), names_to = "domain", names_prefix = "domain_", val

agg_state <- corpus_long |>
  summarize(n_bills = n(), n_mentioned = sum(mentioned, na.rm = TRUE), share_mentioned = n_me

agg_year <- corpus_long |>
  summarize(n_bills = n(), n_mentioned = sum(mentioned, na.rm = TRUE), share_mentioned = n_me

agg_party <- corpus_long |>
  summarize(n_bills = n(), n_mentioned = sum(mentioned, na.rm = TRUE), share_mentioned = n_me

write_csv(agg_state, "data/rq2_state_share.csv")
write_csv(agg_year, "data/rq2_year_share.csv")
write_csv(agg_party, "data/rq2_party_share.csv")
```

6.3 RQ3: High-vs-low domain bills, distinct language and entities

This chunk scores each bill on the six social-determinant domains and defines the high- vs. low-domain groups used in RQ3. For each domain, it counts how many unique dictionary terms and phrases from that domain appear in the bill text, then sums these domain-specific scores into an overall domain_intensity measure per bill. Bills at or above the 75th percentile of domain_intensity are labeled "high," those at or below the 25th percentile are labeled "low," and the remaining bills form a "mid" group that is excluded from contrastive RQ3 analyses.

```
corpus_table <- read_csv("data/corpus_table_with_party_control.csv", show_col_types = FALSE)
domain_dictionary <- read_csv("data/domain_dictionary.csv", show_col_types = FALSE)

domains <- unique(domain_dictionary$domain)
```

```r
for (dom in domains) {
  terms <- domain_dictionary$term_or_phrase[domain_dictionary$domain == dom]
  corpus_table[[paste0("score_", dom)]] <- vapply(
    corpus_table$bill_text,
    function(txt) sum(str_detect(txt, fixed(terms, ignore_case = TRUE))),
    FUN.VALUE = integer(1L)
  )
}

corpus_table_scored <- corpus_table |>
  mutate(
    domain_intensity = rowSums(across(starts_with("score_")))
  )

q <- quantile(corpus_table_scored$domain_intensity, probs = c(0.25, 0.75), na.rm = TRUE)

corpus_table_scored <- corpus_table_scored |>
  mutate(
    domain_group = case_when(
      domain_intensity >= q[2] ~ "high",
      domain_intensity <= q[1] ~ "low",
      TRUE ~ "mid"
    )
  )

write_csv(corpus_table_scored, "data/corpus_table_scored.csv")
saveRDS(corpus_table_scored, "data/corpus_table_scored.rds")

corpus_table_scored
```

```
# A tibble: 3,169 x 22
   bill_id state session_year session_title bill_title bill_text date_introduced
     <dbl> <chr>        <dbl> <chr>         <chr>      <chr>     <date>
 1 1456709 CA            2021 2021-2022 Re~ Health ca~ CALIFORN~ 2021-02-18
 2 1456749 CA            2021 2021-2022 Re~ Medi-Cal:~ CALIFORN~ 2021-02-18
 3 1456763 CA            2021 2021-2022 Re~ Personal ~ CALIFORN~ 2021-02-18
 4 1456779 CA            2021 2021-2022 Re~ Education~ CALIFORN~ 2021-02-18
 5 1456876 CA            2021 2021-2022 Re~ Medi-Cal:~ CALIFORN~ 2021-02-18
 6 1458883 CA            2021 2021-2022 Re~ Mental he~ CALIFORN~ 2021-02-19
 7 1398151 CA            2021 2021-2022 Re~ Mental He~ CALIFORN~ 2021-01-08
 8 1458892 CA            2021 2021-2022 Re~ Mental he~ CALIFORN~ 2021-02-19
 9 1458909 CA            2021 2021-2022 Re~ Perinatal~ CALIFORN~ 2021-02-19
```

```
10 1458913 CA            2021 2021-2022 Re~ Childcare~ CALIFORN~ 2021-02-19
# i 3,159 more rows
# i 15 more variables: chamber <chr>, bill_version_used <chr>,
#   sponsor_party <chr>, governor_party <chr>, upper_party <chr>,
#   lower_party <chr>, party_control_type <chr>,
#   score_economic_stability <int>, score_education_access <int>,
#   score_healthcare_access <int>, score_neighborhood_environment <int>,
#   score_social_community <int>, score_discrimination_equity <int>, ...
```

This chunk implements the unigram part of RQ3a by computing class-based TF-IDF for high- and low-domain bills, treating each group as a single aggregated document and identifying unigrams that are most distinctive to each group. Class-based TF-IDF is appropriate here because the contrast of interest is between groups of bills (high vs. low domain intensity), not between individual bills.

```r
token_table <- read_csv("data/token_table.csv", show_col_types = FALSE)
corpus_table_scored <- readRDS("data/corpus_table_scored.rds")

rq3a_unigrams <- token_table |>
  left_join(corpus_table_scored |> select(bill_id, domain_group), by = "bill_id") |>
  filter(domain_group %in% c("high", "low")) |>
  count(domain_group, token, wt = n, name = "n") |>
  bind_tf_idf(term = token, document = domain_group, n = n)

write_csv(rq3a_unigrams, "data/rq3a_tfidf_unigrams.csv")

rq3a_unigrams
```

```
# A tibble: 39,849 x 6
   domain_group token     n          tf   idf      tf_idf
   <chr>        <chr> <dbl>       <dbl> <dbl>       <dbl>
 1 high         0,000     2 0.000000927 0.693 0.000000642
 2 high         0.005     2 0.000000927 0.693 0.000000642
 3 high         0.03      2 0.000000927 0     0
 4 high         0.05      9 0.00000417  0.693 0.00000289
 5 high         0.1      25 0.0000116   0     0
 6 high         0.10      2 0.000000927 0.693 0.000000642
 7 high         0.13      2 0.000000927 0.693 0.000000642
 8 high         0.138     4 0.00000185  0.693 0.00000128
 9 high         0.15      6 0.00000278  0.693 0.00000193
10 high         0.176     1 0.000000463 0.693 0.000000321
# i 39,839 more rows
```

This chunk extends RQ3a to multi-word phrases by computing class-based TF-IDF for bigrams and trigrams in high- and low-domain bills, so that distinctive phrases are defined by how strongly they characterize each group as a whole rather than individual bills.

```
ngram_table <- read_csv("data/ngram_table.csv", show_col_types = FALSE)
corpus_table_scored <- readRDS("data/corpus_table_scored.rds")

rq3a_ngrams <- ngram_table |>
  left_join(corpus_table_scored |> select(bill_id, domain_group), by = "bill_id") |>
  filter(domain_group %in% c("high", "low")) |>
  count(domain_group, ngram_type, ngram, wt = n, name = "n") |>
  bind_tf_idf(term = ngram, document = domain_group, n = n)

write_csv(rq3a_ngrams, "data/rq3a_tfidf_ngrams.csv")
write_csv(rq3a_ngrams |> filter(ngram_type == "bigram"), "data/rq3a_tfidf_bigrams.csv")
write_csv(rq3a_ngrams |> filter(ngram_type == "trigram"), "data/rq3a_tfidf_trigrams.csv")

rq3a_ngrams
```

```
# A tibble: 236,462 x 7
   domain_group ngram_type ngram                n          tf   idf      tf_idf
   <chr>        <chr>      <chr>            <dbl>       <dbl> <dbl>       <dbl>
 1 high         bigram     0.03 multiplied      2 0.00000190  0.693 0.00000132
 2 high         bigram     0.05 definitions     2 0.00000190  0.693 0.00000132
 3 high         bigram     0.1 definitions      2 0.00000190  0.693 0.00000132
 4 high         bigram     0.138 0.484          2 0.00000190  0.693 0.00000132
 5 high         bigram     0.138 percent        2 0.00000190  0.693 0.00000132
 6 high         bigram     0.176 ounces         1 0.000000950 0.693 0.000000659
 7 high         bigram     0.2 gpr              2 0.00000190  0.693 0.00000132
 8 high         bigram     0.2 percent          2 0.00000190  0.693 0.00000132
 9 high         bigram     0.3 percent          9 0.00000855  0.693 0.00000593
10 high         bigram     0.484 percent        2 0.00000190  0.693 0.00000132
# i 236,452 more rows
```

This chunk supports RQ3b by summarizing named entities in high- and low-domain bills, counting total mentions and bill counts for each entity–type pair, and then restricting attention to ORG and GPE entities. ORG captures organizations and agencies and GPE captures places, which are the entity types most relevant to institutional and geographic reference patterns, whereas LAW was excluded after diagnostics showed it was dominated by internal section/chapter citations and formatting artifacts rather than meaningful named laws.

```r
entity_table <- read_csv("data/entity_table.csv", show_col_types = FALSE)
corpus_table_scored <- readRDS("data/corpus_table_scored.rds")

entity_summary <- entity_table |>
  left_join(corpus_table_scored |> select(bill_id, domain_group), by = "bill_id") |>
  filter(domain_group %in% c("high", "low")) |>
  group_by(domain_group, entity_type, entity) |>
  summarise(
    total_mentions = sum(n, na.rm = TRUE),
    n_bills = n_distinct(bill_id),
    .groups = "drop"
  )

write_csv(entity_summary, "data/rq3b_entity_summary.csv")

entity_summary
```

```
# A tibble: 14,500 x 5
   domain_group entity_type entity             total_mentions n_bills
   <chr>        <chr>       <chr>                       <dbl>   <int>
 1 high         GPE         .68                             2       2
 2 high         GPE         16_states                       2       2
 3 high         GPE         18022(b                         1       1
 4 high         GPE         18_county                       1       1
 5 high         GPE         1_illinois                      1       1
 6 high         GPE         1_ny                            1       1
 7 high         GPE         1_states                        2       2
 8 high         GPE         1_united_states                 3       3
 9 high         GPE         2517                            5       5
10 high         GPE         2972_united_states              1       1
# i 14,490 more rows
```

This chunk extracts the top ORG entities in each group and removes spurious entries that are clearly not true organizations (such as appropriation headers and formatting strings) so that the remaining ORG entities correspond more closely to real institutions.

```r
entity_summary <- read_csv("data/rq3b_entity_summary.csv", show_col_types = FALSE)

top_orgs <- entity_summary |>
  filter(entity_type == "ORG") |>
  group_by(domain_group) |>
```

```r
  slice_max(total_mentions, n = 20, with_ties = FALSE) |>
  ungroup()

low_org_artifacts <- tibble(entity = c(
  "f_l_o_r_i_d_a_h",
  "the_general_revenue_fund",
  "the_committee_on_mental_health_an_act",
  "the_legislature_of_the_state_of_florida_:_section_1",
  "the_legislature_of_the_state_of_florida",
  "no_local_program",
  "the_committee_on_health_an_act",
  "the_mental_health",
  "the_mental_health_and",
  "the_16_department_of_children_and_families",
  "house",
  "the_appropriations_project",
  "printer",
  "the_general_assembly_of_the_commonwealth_of_pennsylvania",
  "appropriations_project_request",
  "the_legislative_reference_bureau"
))

top_orgs_clean <- top_orgs |>
  anti_join(low_org_artifacts, by = "entity")

write_csv(top_orgs_clean, "data/rq3b_top_orgs.csv")

top_orgs_clean
```

```
# A tibble: 24 x 5
   domain_group entity_type entity                          total_mentions n_bills
   <chr>        <chr>       <chr>                                    <dbl>   <dbl>
 1 high         ORG         medicaid                                  1737     294
 2 high         ORG         the_illinois_department                    488      36
 3 high         ORG         the_department_of_human_serv~              453     156
 4 high         ORG         the_department_of_public_hea~              442      83
 5 high         ORG         the_department_of_health                   376      85
 6 high         ORG         medicare                                   329      97
 7 high         ORG         the_interstate_commission                  324      10
 8 high         ORG         the_illinois_state_police                  315      41
 9 high         ORG         mco                                        286       6
10 high         ORG         health_and_safety_code                     273      33
```

```
# i 14 more rows
```

This chunk calculates the high-to-low mention ratio for ORG entities that appear in both groups, highlighting organizations that are disproportionately referenced in high-domain bills.

```r
entity_summary <- read_csv("data/rq3b_entity_summary.csv", show_col_types = FALSE)

org_ratio <- entity_summary |>
  filter(entity_type == "ORG") |>
  select(domain_group, entity, total_mentions) |>
  pivot_wider(
    names_from = domain_group,
    values_from = total_mentions,
    values_fill = 0
  ) |>
  filter(high > 0, low > 0) |>
  mutate(ratio_high_to_low = round(high / low, 2)) |>
  arrange(desc(ratio_high_to_low))

write_csv(org_ratio, "data/rq3b_org_ratio.csv")

org_ratio
```

```
# A tibble: 768 x 4
   entity                          high   low ratio_high_to_low
   <chr>                          <dbl> <dbl>             <dbl>
 1 the_illinois_department          488     1             488
 2 the_nursing_home_care_act        248     2             124
 3 the_internal_revenue_code        111     1             111
 4 the_department_of_children        87     1              87
 5 the_substance_use_disorder_act    72     1              72
 6 medicare                         329     5              65.8
 7 the_department_of_public_health  442     7              63.1
 8 medicaid                        1737    28              62.0
 9 the_mc_/_dd_act                   61     1              61
10 the_department_of_healthcare      60     1              60
# i 758 more rows
```

This chunk identifies the top GPE (place) entities in high- and low-domain bills to explore whether geographic references differ between the two groups.

```r
entity_summary <- read_csv("data/rq3b_entity_summary.csv", show_col_types = FALSE)

top_gpes <- entity_summary |>
  filter(entity_type == "GPE") |>
  group_by(domain_group) |>
  slice_max(total_mentions, n = 20, with_ties = FALSE) |>
  ungroup()

write_csv(top_gpes, "data/rq3b_top_gpes.csv")

top_gpes
```

```
# A tibble: 40 x 5
   domain_group entity_type entity            total_mentions n_bills
   <chr>        <chr>       <chr>                      <dbl>   <dbl>
 1 high         GPE         new_york                    1372     172
 2 high         GPE         illinois                    1310     183
 3 high         GPE         florida                     1116     101
 4 high         GPE         the_united_states           1033     216
 5 high         GPE         f.s.                         819      93
 6 high         GPE         washington                   561      78
 7 high         GPE         ohio                         525      22
 8 high         GPE         united_states                385     106
 9 high         GPE         georgia                      209      36
10 high         GPE         california                   140      39
# i 30 more rows
```

This chunk implements RQ3c by measuring how often high- and low-domain bills reference four institutional categories—Medicaid, schools, local government, and housing—using both n-gram pattern matching and entity-based matching, and summarizing the share of bills with at least one reference in each group. The regex patterns in reference_patterns are designed to capture common lexical variants within each category (for example, medicaid, medi.cal, and managed care for Medicaid, or school, board of education, and school district for schools) while avoiding overly generic terms that would inflate false positives.

```r
ngram_table <- read_csv("data/ngram_table.csv", show_col_types = FALSE)
entity_table <- read_csv("data/entity_table.csv", show_col_types = FALSE)
corpus_table_scored <- readRDS("data/corpus_table_scored.rds")

reference_patterns <- list(
  Medicaid         = "medicaid|medi.cal|managed care|mco",
```

```
  Schools          = "school|education board|board of education|school district",
  Housing          = "housing authorit|public housing|affordable housing|homeless|shelter",
  Local_Government = "local govern|county govern|municipal|city council|department of social
)

group_totals <- corpus_table_scored |>
  filter(domain_group %in% c("high", "low")) |>
  count(domain_group, name = "n_total")

ngram_grouped <- ngram_table |>
  left_join(corpus_table_scored |> select(bill_id, domain_group), by = "bill_id") |>
  filter(domain_group %in% c("high", "low"))

entity_grouped <- entity_table |>
  left_join(corpus_table_scored |> select(bill_id, domain_group), by = "bill_id") |>
  filter(domain_group %in% c("high", "low"))

ngram_ref_counts <- map_dfr(
  names(reference_patterns),
  function(cat) {
    pat <- reference_patterns[[cat]]
    ngram_grouped |>
      filter(str_detect(ngram, regex(pat, ignore_case = TRUE))) |>
      distinct(domain_group, bill_id) |>
      count(domain_group, name = "n_bills_with_ref") |>
      left_join(group_totals, by = "domain_group") |>
      mutate(category = cat, share = n_bills_with_ref / n_total, method = "ngram")
  }
)

entity_ref_counts <- map_dfr(
  names(reference_patterns),
  function(cat) {
    pat <- reference_patterns[[cat]]
    matched <- entity_grouped |>
      filter(str_detect(entity, regex(pat, ignore_case = TRUE))) |>
      distinct(domain_group, bill_id)
    if (nrow(matched) == 0) return(tibble())
    matched |>
      count(domain_group, name = "n_bills_with_ref") |>
      left_join(group_totals, by = "domain_group") |>
      mutate(category = cat, share = n_bills_with_ref / n_total, method = "entity")
```

```
  }
)

reference_summary <- bind_rows(ngram_ref_counts, entity_ref_counts)

write_csv(reference_summary, "data/rq3c_reference_summary.csv")

reference_summary
```

```
# A tibble: 15 x 6
   domain_group n_bills_with_ref n_total category           share method
   <chr>                   <int>   <int> <chr>              <dbl> <chr>
 1 high                      332     901 Medicaid         0.368   ngram
 2 low                        18    1246 Medicaid         0.0144  ngram
 3 high                      352     901 Schools          0.391   ngram
 4 low                       174    1246 Schools          0.140   ngram
 5 high                      165     901 Housing          0.183   ngram
 6 low                        15    1246 Housing          0.0120  ngram
 7 high                      279     901 Local_Government 0.310   ngram
 8 low                        65    1246 Local_Government 0.0522  ngram
 9 high                      317     901 Medicaid         0.352   entity
10 low                        13    1246 Medicaid         0.0104  entity
11 high                      117     901 Schools          0.130   entity
12 low                        51    1246 Schools          0.0409  entity
13 high                       16     901 Housing          0.0178  entity
14 high                        6     901 Local_Government 0.00666 entity
15 low                         4    1246 Local_Government 0.00321 entity
```

7 RQ4: Latent Topics and Domain Linkages

7.1 Topic table

I chose k = 20 after testing several values of k and plotting k against perplexity (not shown in this document). The steepest perplexity drop occurs between k=10 and k=15, with the rate of decrease visibly slowing after 20. More substantively, 20 topics provided enough granularity to capture variation across states and time for RQ4 without producing topics too narrow to interpret.

```r
token_table_lemma <- read_csv("data/token_table_lemma.csv", show_col_types = FALSE)

# Use lemmatized tokens for topic modeling input
word_counts <- token_table_lemma %>%
  rename(token = lemma)

bills_dtm <- word_counts %>%
  cast_dtm(bill_id, token, n)

# Remove rows with all zeros (bills that lost all tokens after cleaning)
bills_dtm <- bills_dtm[rowSums(as.matrix(bills_dtm)) > 0, ]


bills_lda <- LDA(bills_dtm,
                 k = 20,
                 control = list(seed = 1234))

bills_lda  # confirm: "A LDA/VEM topic model with 20 topics"

saveRDS(bills_lda, "data/bills_lda_model.rds")

# Top 10 terms per topic - inspect for substantive coherence
tidy(bills_lda, matrix = "beta") %>%
  group_by(topic) %>%
  slice_max(beta, n = 10) %>%
  ungroup() %>%
  mutate(term = reorder_within(term, beta, topic)) %>%
  ggplot(aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~topic, scales = "free_y") +
  scale_y_reordered() +
  labs(title = "Top 10 Terms per Topic", x = "Beta", y = NULL)
```

I deemed topics 5, 6, 7, 14, 17, and 20 to be non-substantive that mainly contained state-specific formatting. Topic 17 was particularly egregious, as it contained purely markup-related terms that were clearly not meaningful content signals.

7.1.1 Create the final topic table

In this chunk, I assign descriptive labels to the substantive topics and label the six procedural/formatting artifact topics as "procedural_artifact." I also create a binary is_artifact

variable to flag these non-substantive topics for exclusion in RQ4 analyses. The resulting topic_table contains document–topic proportions (gamma) for each bill and topic, along with the assigned topic labels and artifact flags.

```
topic_labels <- tibble(
  topic = 1:20,
  topic_label = c(
    "Involuntary & Inpatient Treatment",   # 1
    "Drug Crime & Penalties",              # 2
    "Maternal & Prenatal Health",          # 3
    "General Regulatory Procedures",       # 4
    "procedural_artifact",                 # 5 - bill digest headers
    "procedural_artifact",                 # 6 - Florida Statutes format
    "procedural_artifact",                 # 7 - Pennsylvania boilerplate
    "Healthcare Facilities & Licensing",   # 8
    "Advisory Bodies & Task Forces",       # 9
    "Workforce Licensing",                 # 10
    "Policy Equity & Conditions",          # 11
    "Budget & Fiscal Provisions",          # 12
    "Licensing & Privacy Records",         # 13
    "procedural_artifact",                 # 14 - Illinois ILCS format
    "Corporate & Regulatory",              # 15
    "Insurance & Medicaid Coverage",       # 16
    "procedural_artifact",                 # 17 - markup artifacts
    "Education & Community",               # 18
    "Criminal Justice",                    # 19
    "procedural_artifact"                  # 20 - Wisconsin LRB format
  ),
  is_artifact = c(
    FALSE, FALSE, FALSE, FALSE,
    TRUE, TRUE, TRUE,
    FALSE, FALSE, FALSE, FALSE, FALSE, FALSE,
    TRUE,
    FALSE,
    FALSE, TRUE, FALSE, FALSE, TRUE
  )
)

topic_table <- tidy(bills_lda, matrix = "gamma") %>%
  rename(
    bill_id = document,
    topic   = topic,
    gamma   = gamma
```

```
  ) %>%
  mutate(bill_id = as.numeric(bill_id)) %>%
  left_join(topic_labels, by = "topic")

write_csv(topic_table, "data/topic_table.csv")
saveRDS(topic_table, "data/topic_table.rds")
saveRDS(bills_lda, "data/bills_lda_model.rds")
write_csv(topic_labels, "data/topic_labels.csv")
```

7.2 4.1 Load topic model and scored corpus

This setup chunk prepares the combined topic–metadata table used in all RQ4 analyses. It joins the document–topic proportions from the fitted LDA model to the scored corpus (state, session year, overall domain intensity, and per-domain scores), creating topic_table_meta for use in RQ4a–RQ4c.

```
topic_table     <- read_csv("data/topic_table.csv", show_col_types = FALSE)
corpus_scored   <- readRDS("data/corpus_table_scored.rds")

# Join topic proportions to bill-level metadata + domain scores
topic_table_meta <- topic_table |>
  left_join(
    corpus_scored |>
      select(bill_id, state, session_year, domain_intensity, domain_group,
             starts_with("score_")),
    by = "bill_id"
  )

# Quick dimension check (should be 3,169 * 20 = 63,380 rows)
nrow(topic_table_meta)

write_csv(topic_table_meta, "data/topic_table_meta.csv")
saveRDS(topic_table_meta, "data/topic_table_meta.rds")
```

7.3 4.2 RQ4a: Topic prevalence

This chunk computes the summary tables used for RQ4a. topic_prevalence records the mean and median document–topic proportion and number of bills for each topic, while dominant_-topic_counts identifies, among substantive topics, how many bills each topic is the dominant topic for (highest γ per bill).

```r
topic_table_meta <- read_csv("data/topic_table_meta.csv", show_col_types = FALSE)

topic_prevalence <- topic_table_meta |>
  group_by(topic, topic_label, is_artifact) |>
  summarise(
    mean_gamma   = mean(gamma, na.rm = TRUE),
    median_gamma = median(gamma, na.rm = TRUE),
    n_bills      = n_distinct(bill_id),
    .groups = "drop"
  ) |>
  arrange(desc(mean_gamma))

# Dominant topic per bill (substantive topics only)
dominant_topic_counts <- topic_table_meta |>
  filter(!is_artifact) |>
  group_by(bill_id) |>
  slice_max(gamma, n = 1, with_ties = FALSE) |>
  ungroup() |>
  count(topic, topic_label, name = "n_dominant_bills") |>
  arrange(desc(n_dominant_bills))

write_csv(topic_prevalence,      "data/rq4_topic_prevalence.csv")
write_csv(dominant_topic_counts, "data/rq4a_dominant_topic_counts.csv")
```

7.4 4.3 RQ4b: Topic–domain co-occurrence

This chunk constructs the topic–domain co-occurrence table for RQ4b by comparing mean document–topic proportions in focal-domain "high" bills to corpus-wide means. For each of the three focal domains, it defines a high-domain subset (any non-zero score for discrimination and neighborhood; top quartile for education), computes mean γ for each topic within that subset, and subtracts the corpus mean γ to obtain gamma_diff values used in the RQ4b heatmap.

```r
topic_table_meta <- read_csv("data/topic_table_meta.csv", show_col_types = FALSE)
corpus_scored   <- readRDS("data/corpus_table_scored.rds")

focal_scores <- c(
  neighborhood_environment = "score_neighborhood_environment",
  education_access         = "score_education_access",
  discrimination_equity    = "score_discrimination_equity"
)
```

```r
# Corpus-wide mean  for substantive topics
corpus_mean_gamma <- topic_table_meta |>
  filter(!is_artifact) |>
  group_by(topic, topic_label) |>
  summarise(corpus_mean_gamma = mean(gamma, na.rm = TRUE), .groups = "drop")

mean_gamma_diff <- purrr::map_dfr(
  names(focal_scores),
  function(dom) {
    score_col <- focal_scores[[dom]]

    if (dom %in% c("discrimination_equity", "neighborhood_environment")) {
      # Sparse domains: any non-zero score is "high"
      high_ids <- corpus_scored |>
        filter(.data[[score_col]] >= 1) |>
        pull(bill_id)
    } else {
      # Education_access: use 75th percentile rule
      q75 <- quantile(corpus_scored[[score_col]], 0.75, na.rm = TRUE)
      high_ids <- corpus_scored |>
        filter(.data[[score_col]] >= q75) |>
        pull(bill_id)
    }

    topic_table_meta |>
      filter(!is_artifact, bill_id %in% high_ids) |>
      group_by(topic, topic_label) |>
      summarise(high_mean_gamma = mean(gamma, na.rm = TRUE), .groups = "drop") |>
      left_join(corpus_mean_gamma, by = c("topic", "topic_label")) |>
      mutate(
        domain     = dom,
        gamma_diff = high_mean_gamma - corpus_mean_gamma
      )
  }
)

write_csv(mean_gamma_diff, "data/rq4b_mean_gamma_diff.csv")
```

7.5 4.4 RQ4c: State and session-year variation

This chunk builds the state- and year-level topic prevalence tables used in RQ4c. state_topic_gamma records mean document–topic proportions and bill counts by state and topic, and year_topic_gamma does the same by session year and topic, providing the inputs for the state heatmap and time-trend figures.

```
topic_table_meta <- read_csv("data/topic_table_meta.csv", show_col_types = FALSE)

state_topic_gamma <- topic_table_meta |>
  filter(!is_artifact, !is.na(state)) |>
  group_by(state, topic, topic_label) |>
  summarise(
    mean_gamma = mean(gamma, na.rm = TRUE),
    n_bills    = n_distinct(bill_id),
    .groups = "drop"
  )

year_topic_gamma <- topic_table_meta |>
  filter(!is_artifact, !is.na(session_year)) |>
  group_by(session_year, topic, topic_label) |>
  summarise(
    mean_gamma = mean(gamma, na.rm = TRUE),
    n_bills    = n_distinct(bill_id),
    .groups = "drop"
  )

write_csv(state_topic_gamma, "data/rq4c_state_topic_gamma.csv")
write_csv(year_topic_gamma,  "data/rq4c_year_topic_gamma.csv")
```

8 Evaluation, Plots, and Output Checks

This section loads the saved analysis tables and generates the figures referenced in the project paper. For each research question, the plotting chunks take the corresponding RQ outputs from the data/ directory, apply a consistent theme and color scheme, and write publication-quality PNG files to output/figures.

8.1 Base Theme

```
GRP_COLORS <- c(high = "#2166ac", low = "#d1495b")
GRP_LABELS <- c(high = "High-Domain Bills", low = "Low-Domain Bills")
BASE_THEME <- theme_minimal(base_size = 11) +
  theme(
    strip.text         = element_text(face = "bold", size = 10),
    axis.title         = element_text(size = 10),
    legend.position    = "bottom",
    plot.title         = element_text(face = "bold", size = 13),
    plot.subtitle      = element_text(size = 10, color = "gray40"),
    panel.grid.major.y = element_blank()
  )

if (!dir.exists("output/figures")) dir.create("output/figures", recursive = TRUE)

saveRDS(GRP_COLORS, "data/plot_colors.rds")
saveRDS(GRP_LABELS, "data/plot_labels.rds")
saveRDS(BASE_THEME, "data/plot_theme.rds")
```

8.2 RQ1 Visualizations

8.2.1 RQ1a TF plot

This chunk visualizes RQ1a by plotting the top 15 dictionary-matched unigrams per domain, using term frequency (TF) from rq1a_tf_terms to show which domain terms appear most often across bills.

```
rq1a_tf_terms <- readr::read_csv("data/rq1a_tf_terms.csv", show_col_types = FALSE)

rq1a_plot <- rq1a_tf_terms |>
  dplyr::group_by(domain) |>
  dplyr::slice_max(tf, n = 15) |>
  dplyr::ungroup() |>
  dplyr::mutate(token = forcats::fct_reorder(token, tf)) |>
  ggplot2::ggplot(ggplot2::aes(tf, token, fill = domain)) +
  ggplot2::geom_col(show.legend = FALSE) +
  ggplot2::facet_wrap(~domain, scales = "free_y") +
  ggplot2::labs(
    title = "Top terms by frequency (RQ1a)",
```

```
    x = "Frequency (TF)",
    y = NULL
  )

ggplot2::ggsave("output/figures/rq1a_tf.png", rq1a_plot,
                width = 10, height = 8, dpi = 300)
```

8.2.2 RQ1b TF-IDF plot

This chunk visualizes RQ1b by plotting the top 15 dictionary-matched lemmas per domain ranked by mean TF-IDF, highlighting which lemmatized terms are most distinctive to each domain rather than simply most frequent.

```
rq1b_tfidf_terms <- readr::read_csv("data/rq1b_tfidf_terms.csv", show_col_types = FALSE)

rq1b_plot <- rq1b_tfidf_terms |>
  dplyr::group_by(domain) |>
  dplyr::slice_max(mean_tfidf, n = 15) |>
  dplyr::ungroup() |>
  dplyr::mutate(lemma = forcats::fct_reorder(lemma, mean_tfidf)) |>
  ggplot2::ggplot(ggplot2::aes(mean_tfidf, lemma, fill = domain)) +
  ggplot2::geom_col(show.legend = FALSE) +
  ggplot2::facet_wrap(~domain, scales = "free_y") +
  ggplot2::labs(
    title = "Top distinctive terms by TF-IDF (RQ1b)",
    x = "Mean TF-IDF",
    y = NULL
  )

ggplot2::ggsave("output/figures/rq1b_tfidf.png", rq1b_plot,
                width = 10, height = 8, dpi = 300)
```

8.2.3 RQ1c n-gram plot

This chunk visualizes the RQ1c bigram results by plotting the top 10 domain-matched bigrams within each domain, showing which two-word phrases occur most frequently in bills associated with each social-determinant domain.

```r
rq1c_bigrams <- readr::read_csv("data/rq1c_bigrams.csv", show_col_types = FALSE)

rq1c_bigrams_plot <- rq1c_bigrams |>
  dplyr::group_by(domain) |>
  dplyr::slice_max(count, n = 10) |>
  dplyr::ungroup() |>
  dplyr::mutate(ngram = forcats::fct_reorder(ngram, count)) |>
  ggplot2::ggplot(ggplot2::aes(count, ngram, fill = domain)) +
  ggplot2::geom_col(show.legend = FALSE) +
  ggplot2::facet_wrap(~domain, scales = "free_y") +
  ggplot2::labs(
    title = "Top bigrams by domain (RQ1c)",
    x = "Count",
    y = NULL
  )

ggplot2::ggsave("output/figures/rq1c_bigrams.png", rq1c_bigrams_plot,
                width = 10, height = 8, dpi = 300)
```

8.3 RQ2 Visualizations

8.3.1 RQ2a state x domain bar plot

This chunk visualizes RQ2a by plotting, for each state, the share of bills that mention each social-determinant domain, using rq2_state_share as input and expressing the shares as percentages.

```r
rq2_state_share <- readr::read_csv("data/rq2_state_share.csv", show_col_types = FALSE)

rq2_state_plot <- rq2_state_share |>
  dplyr::mutate(state = forcats::fct_reorder(state, share_mentioned)) |>
  ggplot2::ggplot(ggplot2::aes(x = state, y = share_mentioned, fill = domain)) +
  ggplot2::geom_col(position = "dodge") +
  ggplot2::scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  ggplot2::labs(
    title = "Share of bills mentioning each domain, by state (RQ2a)",
    x = "State",
    y = "Share of bills",
    fill = "Domain"
  ) +
  ggplot2::theme_minimal()
```

```
ggplot2::ggsave("output/figures/rq2_state_share.png", rq2_state_plot,
                width = 9, height = 6, dpi = 300)
```

8.3.2 RQ2c party control x domain bar plot

This chunk visualizes RQ2c by plotting, for each party-control type, the share of bills that mention each social-determinant domain, using rq2_party_share and ordering party control levels as Democratic, Divided, and Republican for readability.

```
rq2_party_share <- readr::read_csv("data/rq2_party_share.csv", show_col_types = FALSE)

rq2_party_plot <- rq2_party_share |>
  dplyr::mutate(party_control_type = forcats::fct_relevel(
    party_control_type, "Democratic", "Divided", "Republican"
  )) |>
  ggplot2::ggplot(ggplot2::aes(x = party_control_type, y = share_mentioned, fill = domain)) +
  ggplot2::geom_col(position = "dodge") +
  ggplot2::scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  ggplot2::labs(
    title = "Share of bills mentioning each domain, by party control (RQ2c)",
    x = "Party control",
    y = "Share of bills",
    fill = "Domain"
  ) +
  ggplot2::theme_minimal()

ggplot2::ggsave("output/figures/rq2_party_share.png", rq2_party_plot,
                width = 8, height = 6, dpi = 300)
```

8.4 RQ3 Visualizations

This chunk visualizes the unigram part of RQ3a by applying class-based TF-IDF to the high- and low-domain groups, filtering out numeric and code-citation tokens, and plotting the 15 most distinctive alphabetic unigrams for each group.

```
tfidf_unigrams <- readr::read_csv("data/rq3a_tfidf_unigrams.csv", show_col_types = FALSE)
GRP_LABELS <- readRDS("data/plot_labels.rds")
GRP_COLORS <- readRDS("data/plot_colors.rds")
BASE_THEME <- readRDS("data/plot_theme.rds")
```

```r
rq3a_unigram_plot <- tfidf_unigrams |>
  # Remove tokens that are purely numeric or code-citation artifacts
  filter(
    str_detect(token, "^[a-z]+$"),          # only lowercase letters
    nchar(token) >= 3,                     # exclude very short tokens
    !token %in% c("apdss", "115.364", "111.32", "175.35", "2929.13", "165.95", "20210hb2050p
  ) |>
  group_by(domain_group) |>
  slice_max(tf_idf, n = 15, with_ties = FALSE) |>
  ungroup() |>
  mutate(
    token     = reorder_within(token, tf_idf, domain_group),
    grp_label = recode(domain_group, !!!GRP_LABELS)
  ) |>
  ggplot(aes(tf_idf, token, fill = domain_group)) +
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = GRP_COLORS) +
  scale_y_reordered() +
  facet_wrap(~ grp_label, scales = "free") +
  labs(
    title    = "Distinctive Unigrams in High vs. Low Domain Bills (RQ3a)",
    subtitle = "Class-based TF\u2013IDF \u00b7 top 15 alphabetic tokens per group",
    x        = "TF\u2013IDF Score",
    y        = NULL
  ) +
  BASE_THEME

ggsave("output/figures/rq3a_distinctive_unigrams.png",
       rq3a_unigram_plot, width = 10, height = 6, dpi = 300)
rq3a_unigram_plot
```

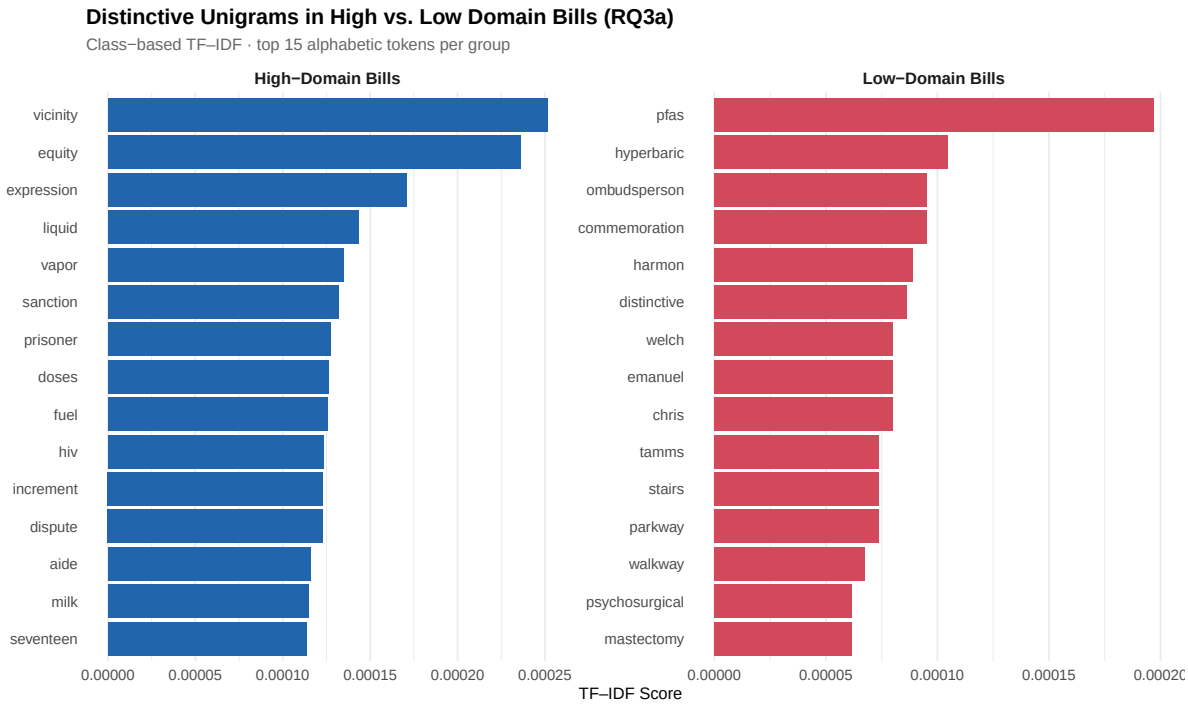

Figure 1: Figure RQ3a. Top 15 most TF-IDF-distinctive unigrams for high-domain and low-domain bills after removing purely numeric and code-citation tokens. Class-based TF-IDF treats each group as a single aggregated document.

This chunk completes RQ3a by plotting the 15 most TF-IDF-distinctive bigrams for high- and low-domain bills, using class-based TF-IDF to highlight multi-word phrases that characterize each group.

```
tfidf_bigrams <- readr::read_csv("data/rq3a_tfidf_bigrams.csv", show_col_types = FALSE)
GRP_LABELS <- readRDS("data/plot_labels.rds")
GRP_COLORS <- readRDS("data/plot_colors.rds")
BASE_THEME <- readRDS("data/plot_theme.rds")

rq3a_bigram_plot <- tfidf_bigrams |>
  group_by(domain_group) |>
  slice_max(tf_idf, n = 15, with_ties = FALSE) |>
  ungroup() |>
  mutate(
    ngram     = reorder_within(ngram, tf_idf, domain_group),
    grp_label = recode(domain_group, !!!GRP_LABELS)
  ) |>
  ggplot(aes(tf_idf, ngram, fill = domain_group)) +
```

```
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = GRP_COLORS) +
  scale_y_reordered() +
  facet_wrap(~ grp_label, scales = "free") +
  labs(
    title    = "Distinctive Bigrams in High vs. Low Domain Bills (RQ3a)",
    subtitle = "Class-based TF\u2013IDF \u00b7 top 15 bigrams per group",
    x        = "TF\u2013IDF Score",
    y        = NULL
  ) +
  BASE_THEME

ggsave("output/figures/rq3a_distinctive_bigrams.png",
       rq3a_bigram_plot, width = 10, height = 6, dpi = 300)
rq3a_bigram_plot
```

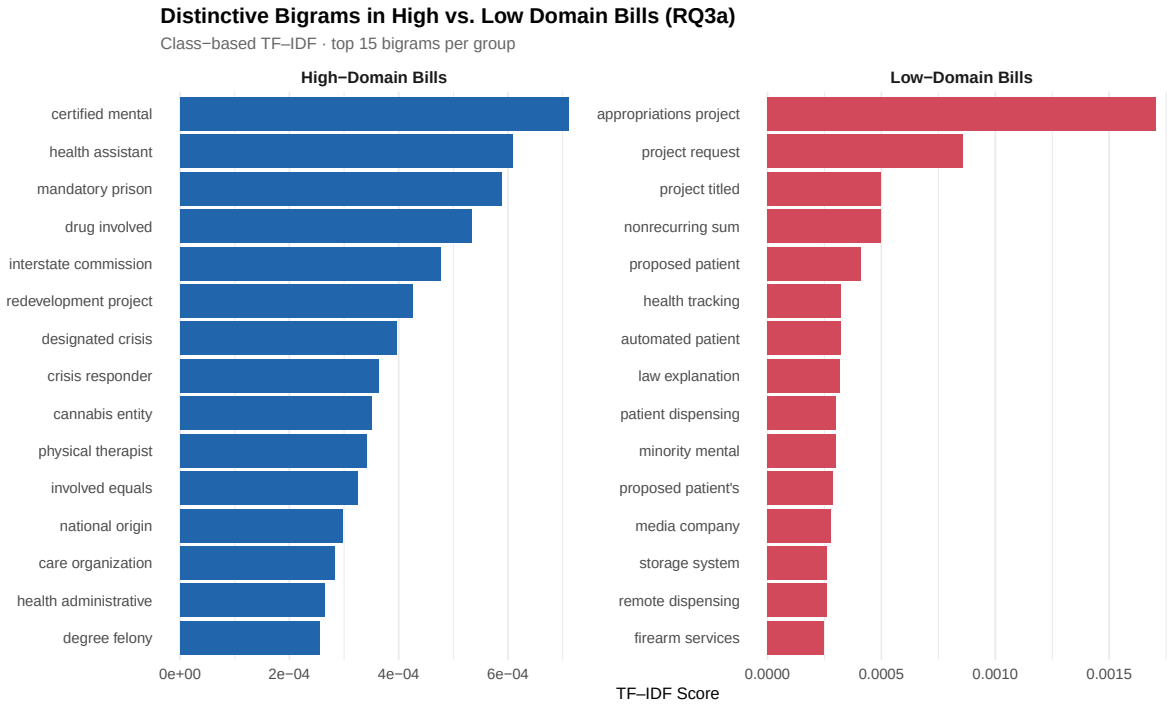

Figure 2: Figure RQ3a-ii. Top 15 most TF-IDF-distinctive bigrams for high-domain and low-domain bills.

This chunk visualizes RQ3b for organizations by plotting the top 15 cleaned ORG entities in high- and low-domain bills, using total mention counts from rq3b_top_orgs.csv and faceting

by domain group.

```
top_orgs_clean <- readr::read_csv("data/rq3b_top_orgs.csv", show_col_types = FALSE)
GRP_LABELS <- readRDS("data/plot_labels.rds")
GRP_COLORS <- readRDS("data/plot_colors.rds")
BASE_THEME <- readRDS("data/plot_theme.rds")

rq3b_org_plot <- top_orgs_clean |>
  mutate(
    entity_label = str_replace_all(entity, "_", " "),
    entity_label = str_to_title(entity_label),
    entity_label = reorder_within(entity_label, total_mentions, domain_group),
    grp_label    = recode(domain_group, !!!GRP_LABELS)
  ) |>
  ggplot(aes(total_mentions, entity_label, fill = domain_group)) +
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = GRP_COLORS) +
  scale_y_reordered() +
  scale_x_continuous(labels = comma) +
  facet_wrap(~ grp_label, scales = "free") +
  labs(
    title    = "Top ORG Named Entities in High vs. Low Domain Bills (RQ3b)",
    subtitle = "Total mentions across all bills in group \u00b7 spaCy en_core_web_lg",
    x        = "Total Mentions",
    y        = NULL
  ) +
  BASE_THEME

ggsave("output/figures/rq3b_top_orgs.png",
       rq3b_org_plot, width = 11, height = 7, dpi = 300)
rq3b_org_plot
```

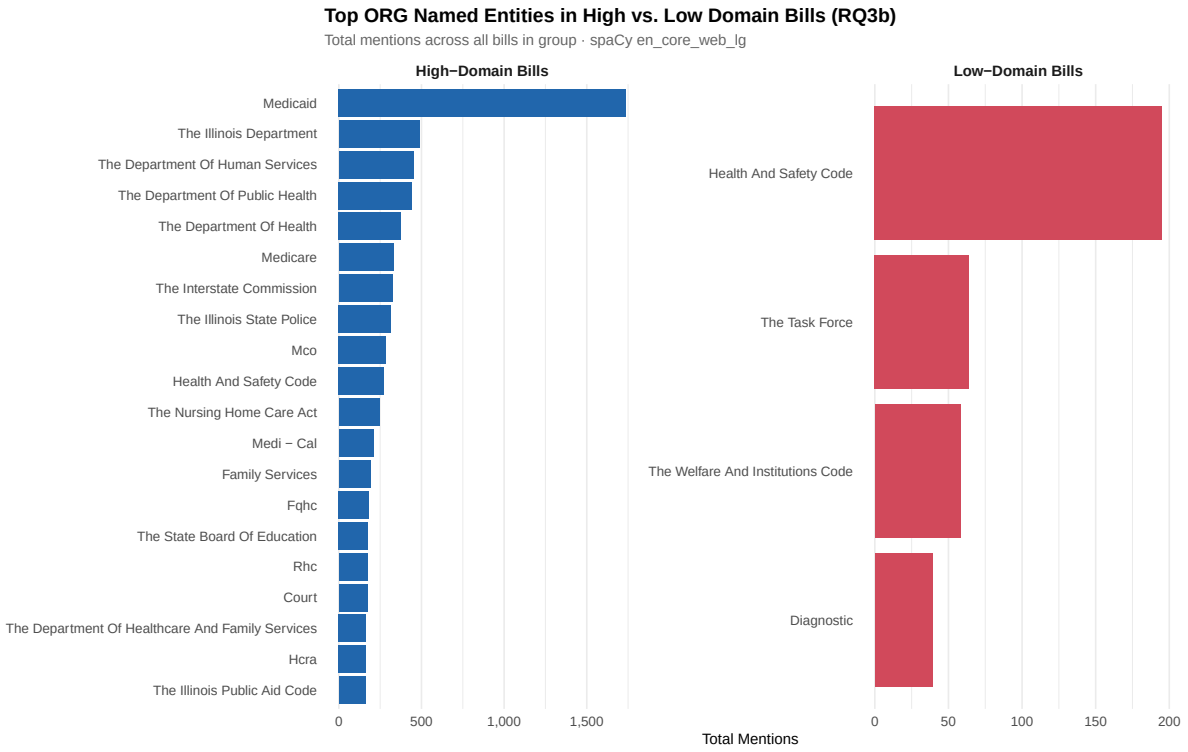

Figure 3: Figure RQ3b. Top 15 ORG named entities in high-domain and low-domain bills by total mention count. Obvious non-organizational header and appropriation strings were removed from the low-domain panel via a group-specific cleaning pass.

This chunk further examines RQ3b by plotting the 20 ORG entities with the highest high-to-low mention ratios (among entities with at least 10 high-group mentions), highlighting organizations that are especially concentrated in high-domain bills.

```
org_ratio <- readr::read_csv("data/rq3b_org_ratio.csv", show_col_types = FALSE)
GRP_COLORS <- readRDS("data/plot_colors.rds")
BASE_THEME <- readRDS("data/plot_theme.rds")

rq3b_ratio_plot <- org_ratio |>
  filter(high >= 10) |>
  slice_max(ratio_high_to_low, n = 20, with_ties = FALSE) |>
  mutate(
    entity_label = str_to_title(str_replace_all(entity, "_", " ")),
    entity_label = fct_reorder(entity_label, ratio_high_to_low)
  ) |>
  ggplot(aes(x = ratio_high_to_low, y = entity_label)) +
```

```
  geom_segment(
    aes(x = 1, xend = ratio_high_to_low, yend = entity_label),
    color = "gray70", linewidth = 0.6
  ) +
  geom_point(size = 3, color = GRP_COLORS["high"]) +
  geom_vline(xintercept = 1, linetype = "dashed", color = "gray40") +
  scale_x_continuous(breaks = seq(0, 500, by = 50)) +
  labs(
    title    = "ORG Entities Most Over-Represented in High-Domain Bills (RQ3b)",
    subtitle = "Ratio = high-group mentions / low-group mentions \u00b7 dashed line = equal r
    x        = "High-to-Low Mention Ratio",
    y        = NULL
  ) +
  BASE_THEME

ggsave("output/figures/rq3b_org_ratio.png",
       rq3b_ratio_plot, width = 9, height = 7, dpi = 300)
rq3b_ratio_plot
```

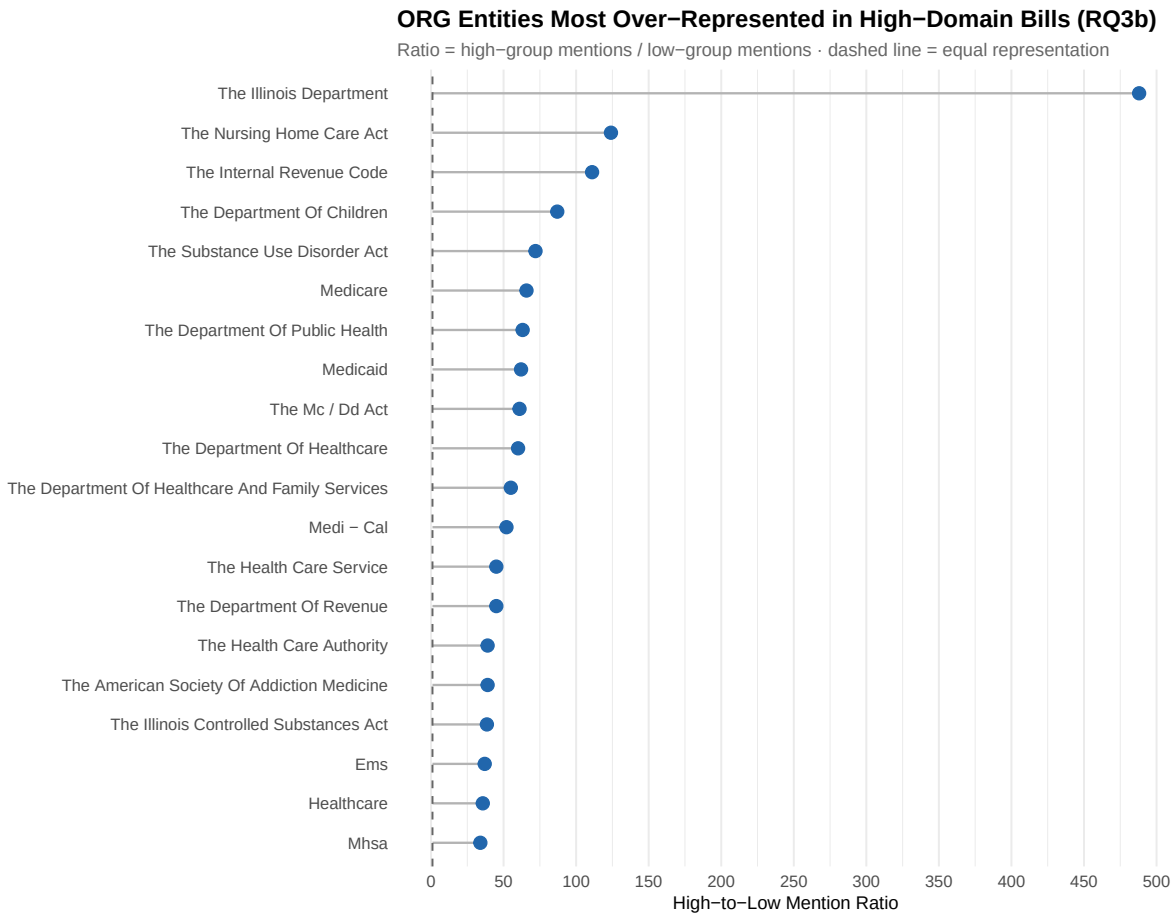

Figure 4: Figure RQ3b-ii. High-to-low mention ratio for ORG entities present in both bill groups (restricted to entities with >=10 high-group mentions). Dashed line marks equal representation (ratio = 1).

This chunk completes the RQ3b visualizations by plotting the top 15 GPE place entities in high- and low-domain bills, using total mention counts from rq3b_top_gpes.csv to show which jurisdictions and locations appear most often in each group.

```
top_gpes <- readr::read_csv("data/rq3b_top_gpes.csv", show_col_types = FALSE)
GRP_LABELS <- readRDS("data/plot_labels.rds")
GRP_COLORS <- readRDS("data/plot_colors.rds")
BASE_THEME <- readRDS("data/plot_theme.rds")

rq3b_gpe_plot <- top_gpes |>
  mutate(
    entity_label = str_to_title(str_replace_all(entity, "_", " ")),
```

```r
    entity_label = reorder_within(entity_label, total_mentions, domain_group),
    grp_label    = recode(domain_group, !!!GRP_LABELS)
  ) |>
  ggplot(aes(total_mentions, entity_label, fill = domain_group)) +
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = GRP_COLORS) +
  scale_y_reordered() +
  scale_x_continuous(labels = comma) +
  facet_wrap(~ grp_label, scales = "free") +
  labs(
    title    = "Top Place Names (GPE) in High vs. Low Domain Bills (RQ3b)",
    subtitle = "Total mentions across all bills in group \u00b7 spaCy en_core_web_lg",
    x        = "Total Mentions",
    y        = NULL
  ) +
  BASE_THEME

ggsave("output/figures/rq3b_top_gpes.png",
       rq3b_gpe_plot, width = 11, height = 7, dpi = 300)
rq3b_gpe_plot
```

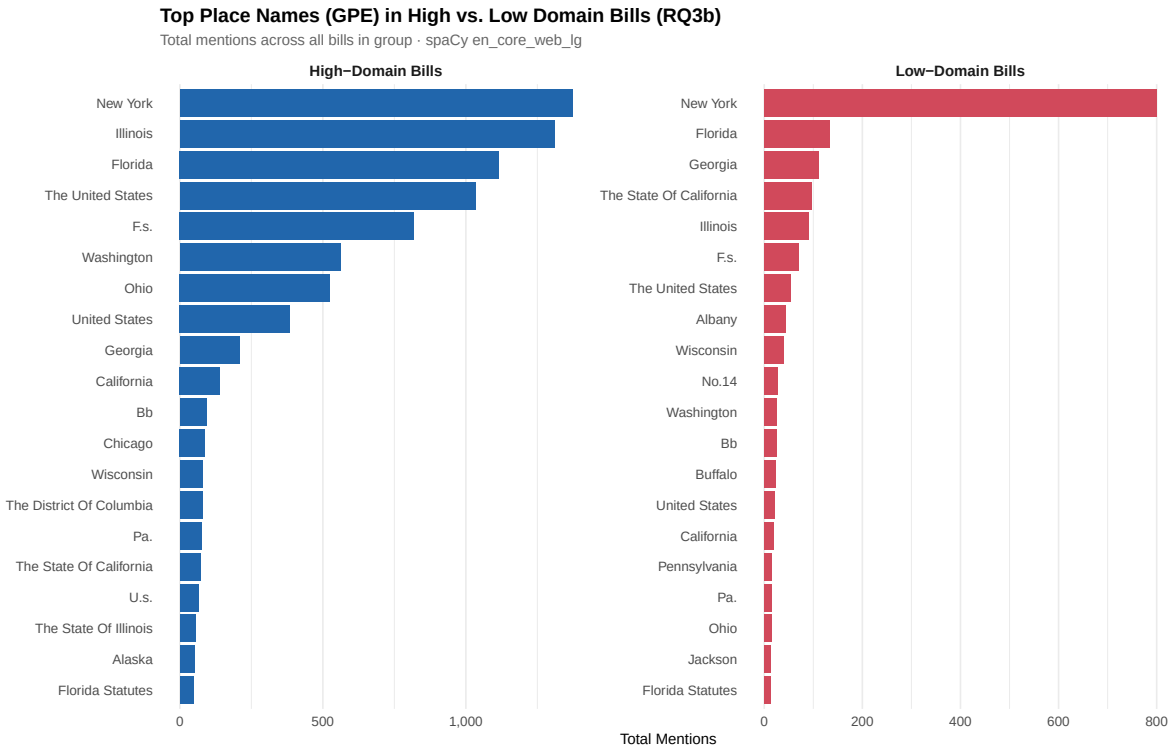

Figure 5: Figure RQ3b-iii. Top 15 GPE (geopolitical) named entities in high-domain and low-domain bills by total mention count.

This chunk visualizes RQ3c by plotting, for high- and low-domain bills, the share of bills that contain at least one reference to Medicaid, schools, housing, or local government, with separate bars for the n-gram and named-entity measures.

```
reference_summary <- readr::read_csv("data/rq3c_reference_summary.csv", show_col_types = FALS
GRP_LABELS <- readRDS("data/plot_labels.rds")
GRP_COLORS <- readRDS("data/plot_colors.rds")
BASE_THEME <- readRDS("data/plot_theme.rds")

# Complete the grid so missing combinations appear as NA (not dropped)
reference_summary_complete <- reference_summary |>
  mutate(category = str_replace_all(category, "_", " ")) |>
  complete(
    category, domain_group, method,
    fill = list(n_bills_with_ref = NA_integer_, n_total = NA_integer_, share = NA_real_)
  ) |>
  mutate(
```

```r
    grp_label    = recode(domain_group, !!!GRP_LABELS),
    method_label = recode(method, ngram = "N-gram", entity = "Named Entity")
  )

rq3c_plot <- reference_summary_complete |>
  ggplot(aes(
    x     = share,
    y     = fct_reorder(category, share, .fun = max, na.rm = TRUE),
    fill  = domain_group,
    alpha = method_label
  )) +
  geom_col(
    position = position_dodge2(width = 0.7, preserve = "single"),
    width = 0.6, na.rm = TRUE
  ) +
  scale_fill_manual(values = GRP_COLORS, labels = GRP_LABELS, name = "Domain Group") +
  scale_alpha_manual(
    values = c("N-gram" = 1.0, "Named Entity" = 0.55),
    name   = "Method",
    na.translate = FALSE
  ) +
  scale_x_continuous(labels = percent_format(accuracy = 1)) +
  labs(
    title    = "Institutional Reference Rates in High vs. Low Domain Bills (RQ3c)",
    subtitle = "Share of bills with \u22651 reference \u00b7 solid = n-gram, faded = named en
    x        = "Share of Bills",
    y        = NULL
  ) +
  BASE_THEME +
  theme(legend.position = "right")

ggsave("output/figures/rq3c_reference_rates.png",
       rq3c_plot, width = 10, height = 6, dpi = 300)
```

```
Warning: `fct_reorder()` removing 1 missing value.
i Use `.na_rm = TRUE` to silence this message.
i Use `.na_rm = FALSE` to preserve NAs.
```

```r
rq3c_plot
```

```
Warning: `fct_reorder()` removing 1 missing value.
```

```
i Use `.na_rm = TRUE` to silence this message.
i Use `.na_rm = FALSE` to preserve NAs.
```

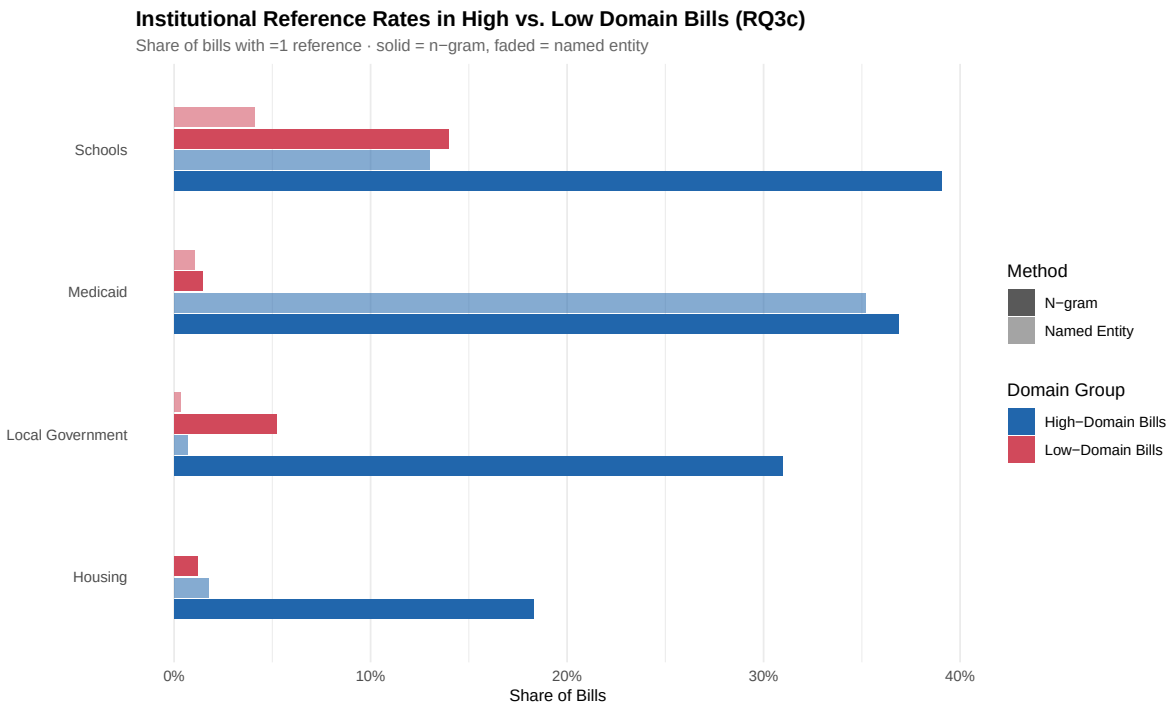

Figure 6: Figure RQ3c. Share of bills containing at least one reference to each institutional category, by domain group and measurement method. The Housing category has no entity-method data point because no housing-authority entities appear in the entity table after cleaning.

8.5 RQ4 Visualizations

This chunk addresses RQ4a by plotting the mean document–topic proportion for each substantive LDA topic across all 3,169 bills, highlighting SDOH-focal topics and excluding topics dominated by boilerplate or citation tokens.

```
topic_prevalence <- readr::read_csv("data/rq4_topic_prevalence.csv", show_col_types = FALSE)
BASE_THEME <- readRDS("data/plot_theme.rds")

# Flag topics as SDOH-focal vs. other for color coding
sdoh_focal_topics <- c(
  "Insurance & Medicaid Coverage",  # maps to healthcare_access
```

```
  "Education & Community",           # maps to education_access
  "Policy Equity & Conditions",      # maps to discrimination_equity
  "Criminal Justice"                 # maps to social_community / nbhd
)

rq4a_plot_data <- topic_prevalence |>
  filter(!is_artifact) |>
  mutate(
    topic_label  = str_wrap(topic_label, width = 28),
    topic_label  = fct_reorder(topic_label, mean_gamma),
    sdoh_focal   = topic_label %in% str_wrap(sdoh_focal_topics, width = 28)
  )

rq4a_mean_gamma_plot <- rq4a_plot_data |>
  ggplot(aes(
    x    = mean_gamma,
    y    = topic_label,
    fill = sdoh_focal
  )) +
  geom_col(width = 0.75) +
  geom_text(
    aes(label = scales::number(mean_gamma, accuracy = 0.001)),
    hjust = -0.15, size = 3, color = "gray30"
  ) +
  scale_fill_manual(
    values = c(`TRUE` = "#4e79a7", `FALSE` = "#b0bec5"),
    labels = c(`TRUE` = "SDOH-focal topic", `FALSE` = "Other substantive topic"),
    name   = NULL
  ) +
  scale_x_continuous(
    expand = expansion(mult = c(0, 0.18)),
    labels = number_format(accuracy = 0.01)
  ) +
  labs(
    title    = "Topic Prevalence in the Mental Health Bill Corpus (RQ4a)",
    subtitle = "Mean document\u2013topic proportion (\u03b3) across 3,169 bills \u00b7 topics
    x        = "Mean \u03b3",
    y        = NULL
  ) +
  BASE_THEME +
  theme(legend.position = "top")
```

```
ggsave("output/figures/rq4a_mean_gamma.png",
       rq4a_mean_gamma_plot, width = 9, height = 6, dpi = 300)
rq4a_mean_gamma_plot
```

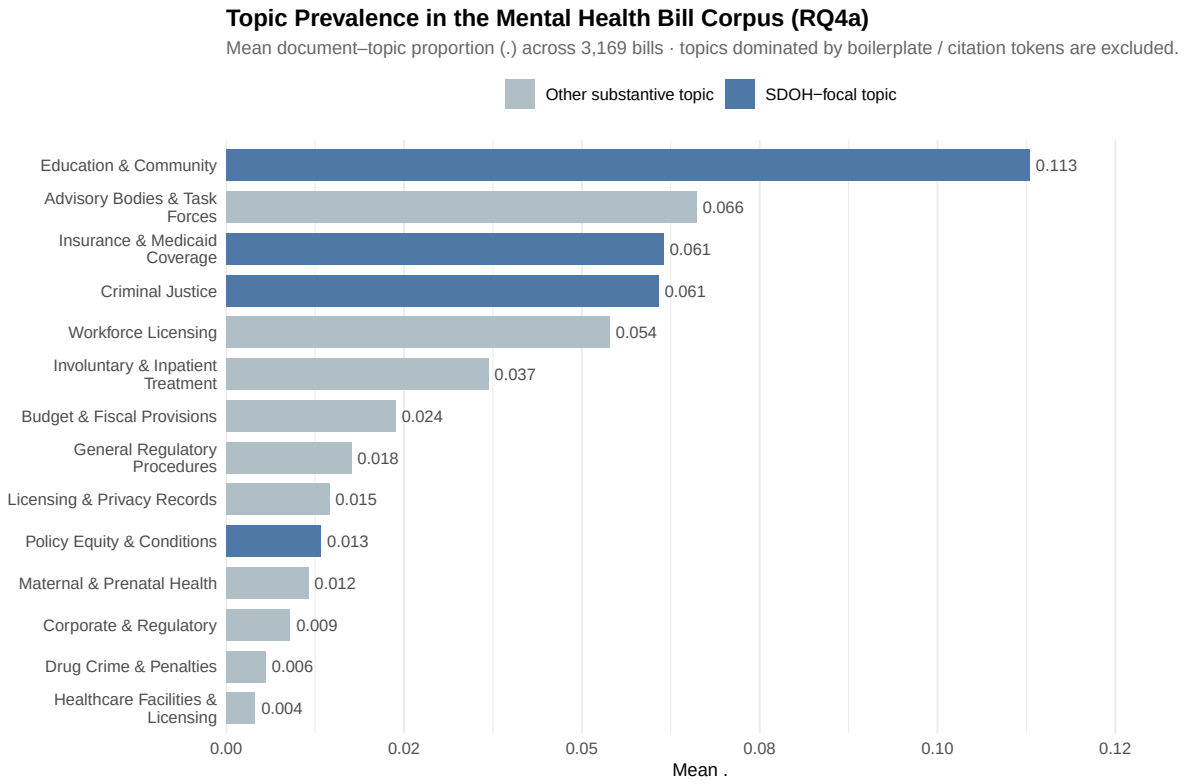

Figure 7: Figure RQ4a-i. Mean document–topic proportion ( ) for each of the 14 substantive LDA topics across the full corpus of 3,169 state mental health bills (2021–2024); topics dominated by boilerplate or numeric junk are excluded.

This companion figure for RQ4a shows, for each substantive topic, how many bills it is the dominant topic for (the topic with the highest document–topic proportion), summarizing how often each topic serves as the primary frame in the corpus.

```
dominant_topic_counts <- readr::read_csv("data/rq4a_dominant_topic_counts.csv",
                                         show_col_types = FALSE)
BASE_THEME <- readRDS("data/plot_theme.rds")

rq4a_dom_plot <- dominant_topic_counts |>
  mutate(
```

```
    topic_label = str_wrap(topic_label, width = 28),
    topic_label = fct_reorder(topic_label, n_dominant_bills)
  ) |>
  ggplot(aes(x = n_dominant_bills, y = topic_label)) +
  geom_col(fill = "#59a14f", width = 0.75) +
  geom_text(
    aes(label = comma(n_dominant_bills)),
    hjust = -0.15, size = 3, color = "gray30"
  ) +
  scale_x_continuous(
    expand = expansion(mult = c(0, 0.18)),
    labels = comma
  ) +
  labs(
    title    = "Dominant-Topic Assignment Counts (RQ4a)",
    subtitle = "Number of bills where each topic has the highest \u03b3 \u00b7 substantive to
    x        = "Number of Bills",
    y        = NULL
  ) +
  BASE_THEME

ggsave("output/figures/rq4a_dominant_topic_counts.png",
       rq4a_dom_plot, width = 9, height = 6, dpi = 300)
rq4a_dom_plot
```

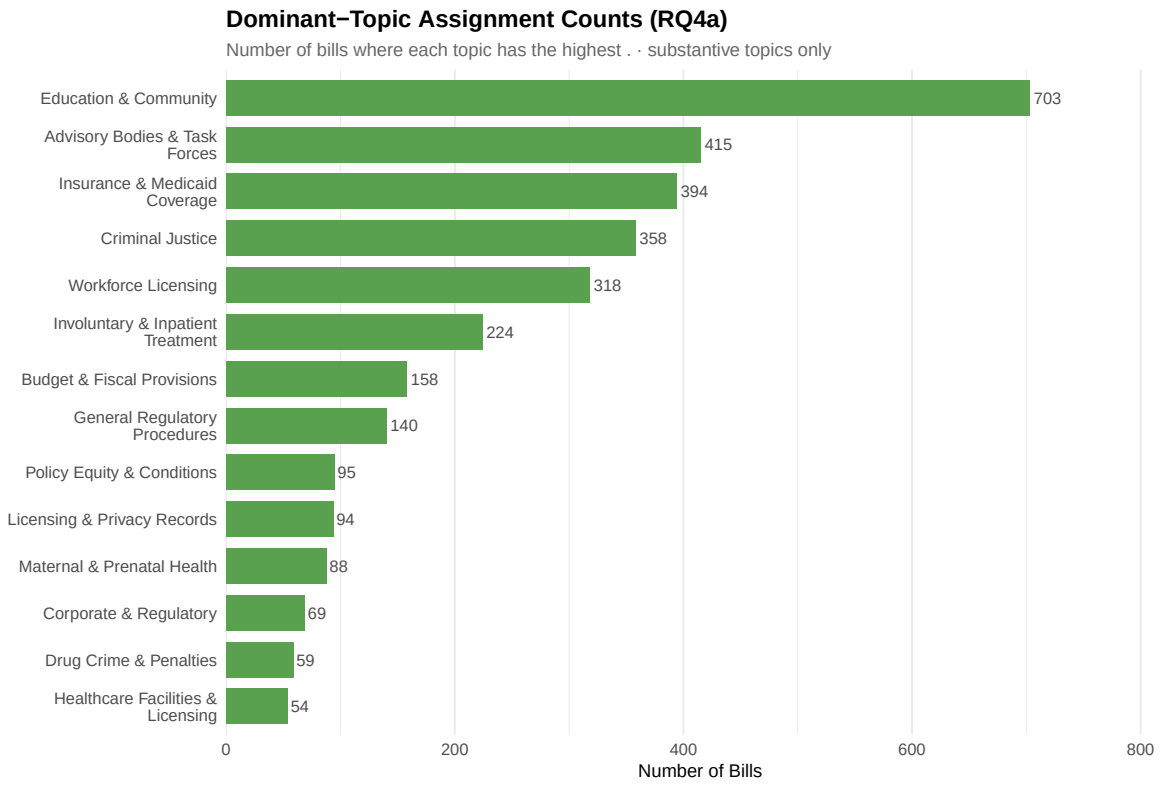

Figure 8: Figure RQ4a-ii. Number of bills for which each substantive topic has the highest document–topic proportion (dominant-topic assignment). Bills with tied maximum gamma are counted once for the first-ranked topic.

This chunk visualizes RQ4b by plotting, for three focal domains, the difference in mean document–topic proportion between high-domain bills and the full corpus for each substantive topic, so that topics over- or under-represented in high-domain bills stand out as blue or red cells.

```
mean_gamma_diff <- readr::read_csv("data/rq4b_mean_gamma_diff.csv", show_col_types = FALSE)
BASE_THEME <- readRDS("data/plot_theme.rds")

heatmap_data <- mean_gamma_diff |>
  mutate(
    topic_label_wrap = str_wrap(topic_label, width = 26),
    domain_label     = recode(
      domain,
      neighborhood_environment = "Neighborhood &\nBuilt Env.",
      education_access         = "Education\nAccess & Quality",
```

```r
      discrimination_equity    = "Discrimination,\nEquity & Rights"
    )
  )

# Order topics by maximum absolute difference across focal domains
topic_order <- heatmap_data |>
  group_by(topic_label_wrap) |>
  summarise(max_abs = max(abs(gamma_diff)), .groups = "drop") |>
  arrange(max_abs) |>
  pull(topic_label_wrap)

heatmap_data <- heatmap_data |>
  mutate(topic_label_wrap = factor(topic_label_wrap, levels = topic_order))

rq4b_heatmap <- heatmap_data |>
  ggplot(aes(
    x    = domain_label,
    y    = topic_label_wrap,
    fill = gamma_diff
  )) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(
    aes(
      label = ifelse(
        abs(gamma_diff) >= 0.005,
        sprintf("%+.3f", gamma_diff),
        ""
      ),
      color = abs(gamma_diff) > 0.012
    ),
    size = 2.9
  ) +
  scale_fill_gradient2(
    low      = "#d1495b",
    mid      = "white",
    high     = "#2166ac",
    midpoint = 0,
    limits   = c(-0.03, 0.03),
    oob      = scales::squish,
    name     = "\u03b3 difference\n(high \u2212 corpus mean)"
  ) +
  scale_color_manual(values = c(`TRUE` = "white", `FALSE` = "gray30"),
```

```
                     guide  = "none") +
  labs(
    title    = "Topic\u2013Domain Co-occurrence: Mean \u03b3 Difference in High-Domain Bills
    subtitle = "High-domain bills = \u226575th percentile of focal domain score \u00b7 topics
    x        = NULL,
    y        = NULL
  ) +
  BASE_THEME +
  theme(
    axis.text.x  = element_text(size = 10, face = "bold"),
    axis.text.y  = element_text(size = 9),
    legend.position = "right",
    panel.grid   = element_blank()
  )

ggsave("output/figures/rq4b_topic_domain_heatmap.png",
       rq4b_heatmap, width = 9, height = 7, dpi = 300)
rq4b_heatmap
```

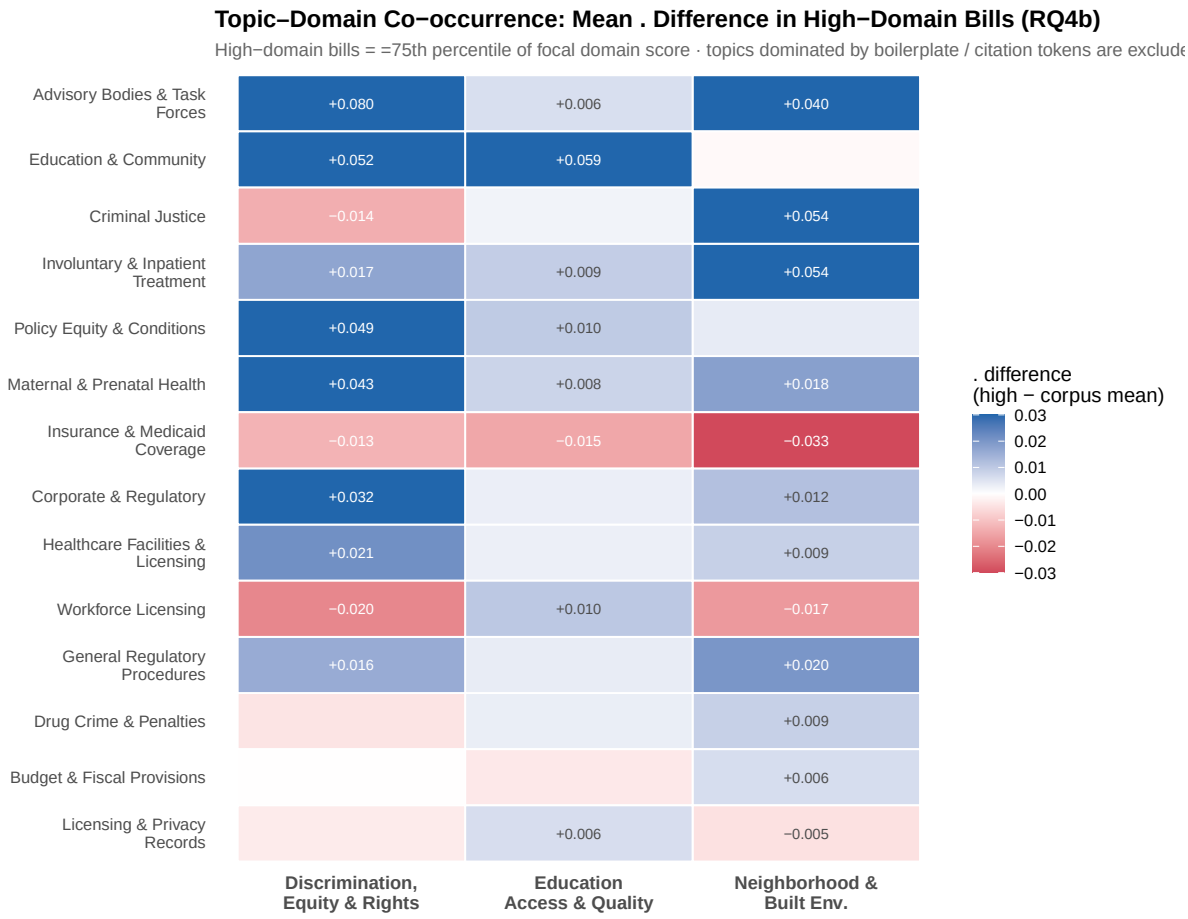

Figure 9: Figure RQ4b. Mean document–topic proportion ( ) difference between high-domain bills (75th percentile of focal domain score) and the full corpus, for each substantive topic and three focal SDOH domains. Positive values (blue) indicate over-representation of the topic in high-domain bills; negative values (red) indicate under-representation.

This companion figure for RQ4b zooms in on the Education & Community topic and the Education Access & Quality domain, plotting bill-level document–topic proportions against education domain scores with a log-linear OLS smooth to show how topic prevalence increases with stronger education framing.

```
topic_table_meta <- readr::read_csv("data/topic_table_meta.csv", show_col_types = FALSE)
BASE_THEME <- readRDS("data/plot_theme.rds")

# Identify the best Education & Community × Education Access pairing
scatter_topic  <- "Education & Community"
```

```r
scatter_domain <- "score_education_access"

scatter_data <- topic_table_meta |>
  filter(!is_artifact, topic_label == scatter_topic) |>
  select(bill_id, state, session_year, gamma,
         domain_score = all_of(scatter_domain))

rq4b_scatter <- scatter_data |>
  ggplot(aes(x = domain_score, y = gamma)) +
  geom_jitter(aes(color = state),
              width = 0.25, height = 0, alpha = 0.45, size = 1.2) +
  geom_smooth(method = "lm", formula = y ~ log1p(x), se = TRUE,
              color = "#2166ac", fill = "#2166ac", alpha = 0.15,
              linewidth = 0.9) +
  scale_color_brewer(palette = "Paired", name = "State") +
  scale_y_continuous(labels = number_format(accuracy = 0.01)) +
  labs(
    title    = paste0("\u03b3 vs. Education Score: \"",
                      scatter_topic, "\" Topic (RQ4b)"),
    subtitle = "Log-linear OLS smooth (y ~ log1p(x)) with 95% CI \u00b7 jitter applied at int
    x        = "Education Access & Quality Domain Score (unique dict. matches)",
    y        = paste0("Document\u2013Topic Proportion (\u03b3): ",
                      scatter_topic)
  ) +
  BASE_THEME +
  theme(legend.position = "right")

ggsave("output/figures/rq4b_gamma_educ_scatter.png",
       rq4b_scatter, width = 8, height = 5, dpi = 300)
rq4b_scatter
```

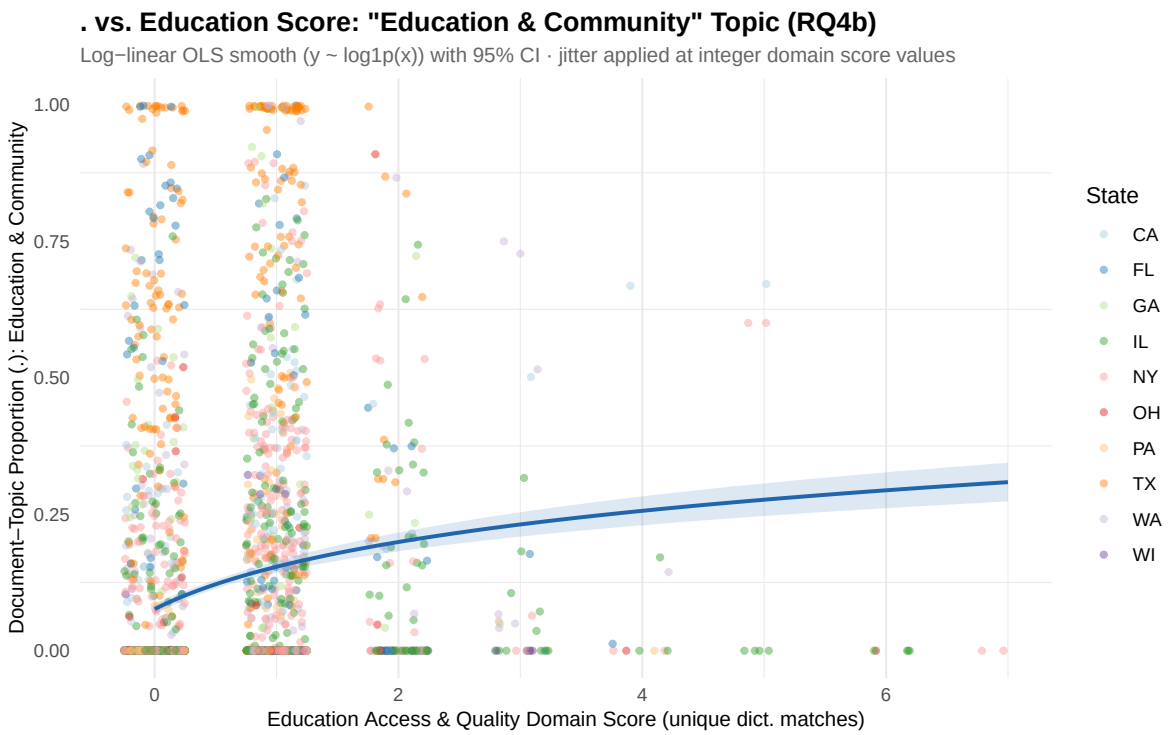

Figure 10: Figure RQ4b-ii. Scatter plot of document–topic proportion ( ) vs. Education Access & Quality domain score for the Education & Community topic. Smooth line is a log-linear OLS fit (y ~ log1p(x)); each point is one bill, jittered horizontally to reduce overplotting at integer domain scores.

This chunk addresses RQ4c by plotting mean document–topic proportions by state and topic, using a heatmap so that states where particular topics are especially prevalent (or rare) are visible as darker or lighter cells.

```
topic_prevalence <- readr::read_csv("data/rq4_topic_prevalence.csv", show_col_types = FALSE)
topic_table_meta <- read_csv("data/topic_table_meta.csv", show_col_types = FALSE)
BASE_THEME <- readRDS("data/plot_theme.rds")

state_topic_gamma <- topic_table_meta |>
  filter(!is_artifact, !is.na(state)) |>
  group_by(state, topic, topic_label) |>
  summarise(
    mean_gamma = mean(gamma, na.rm = TRUE),
    n_bills    = n_distinct(bill_id),
    .groups    = "drop"
  )
```

```r
# Order topics by corpus-wide mean gamma (highest at top)
topic_order_corpus <- topic_prevalence |>
  filter(!is_artifact) |>
  arrange(desc(mean_gamma)) |>
  pull(topic_label)

# Wrap labels for readability
state_topic_heatmap_data <- state_topic_gamma |>
  mutate(
    topic_label_wrap = str_wrap(topic_label, width = 24),
    topic_label_wrap = factor(
      topic_label_wrap,
      levels = rev(str_wrap(topic_order_corpus, width = 24))
    )
  )

rq4c_state_heatmap <- state_topic_heatmap_data |>
  ggplot(aes(x = state, y = topic_label_wrap, fill = mean_gamma)) +
  geom_tile(color = "white", linewidth = 0.4) +
  geom_text(
    aes(
      label = sprintf("%.3f", mean_gamma),
      color = mean_gamma > 0.09
    ),
    size = 2.5
  ) +
  scale_fill_gradient(
    low  = "#f7fbff",
    high = "#08306b",
    name = "Mean \u03b3"
  ) +
  scale_color_manual(
    values = c(`TRUE` = "white", `FALSE` = "gray30"),
    guide  = "none"
  ) +
  labs(
    title    = "State \u00d7 Topic Prevalence Heatmap (RQ4c)",
    subtitle = "Mean document\u2013topic proportion (\u03b3) per state \u00b7 topics dominate
    x        = "State",
    y        = NULL
  ) +
  BASE_THEME +
```

```
  theme(
    axis.text.x     = element_text(size = 10, face = "bold"),
    axis.text.y     = element_text(size = 9),
    legend.position = "right",
    panel.grid      = element_blank()
  )

ggsave("output/figures/rq4c_state_topic_heatmap.png",
       rq4c_state_heatmap, width = 12, height = 7, dpi = 300)
rq4c_state_heatmap
```

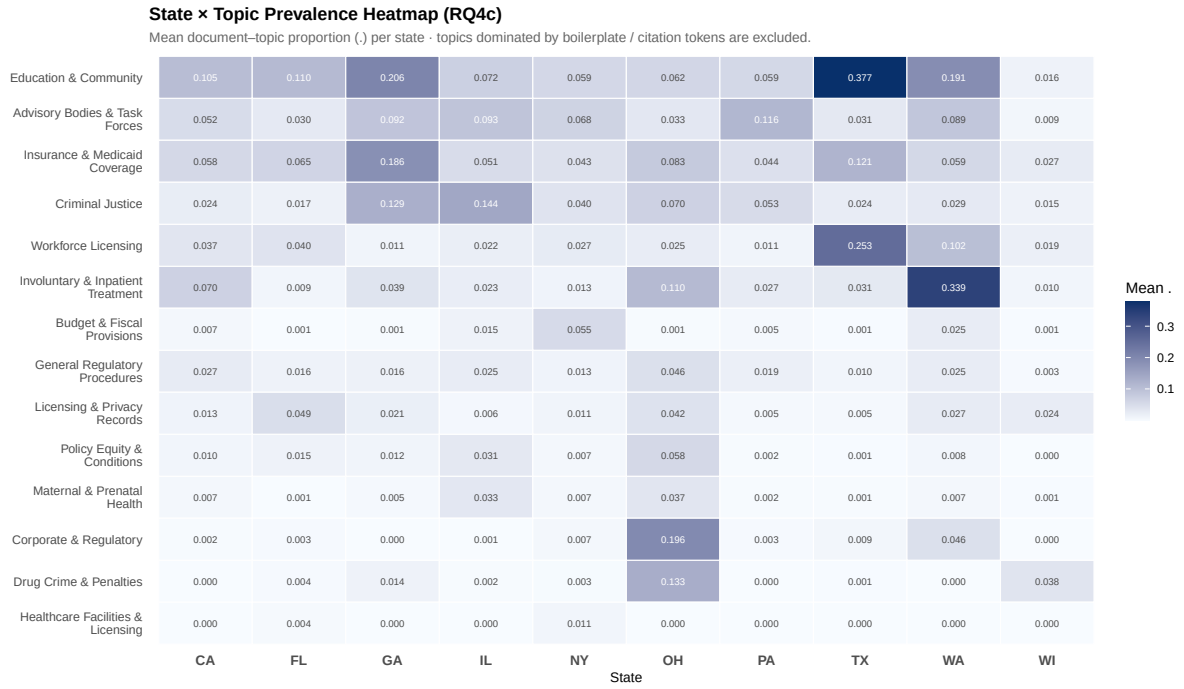

Figure 11: Figure RQ4c-i. Mean document–topic proportion ( ) by state and substantive topic. Each cell represents the mean across all bills introduced in that state. Rows are topics ordered by corpus-wide mean ; columns are the ten study states.

This chunk continues RQ4c by plotting mean document–topic proportions over session years for the four SDOH-focal topics, showing how the prevalence of each focal topic changes between 2021 and 2023.

```
topic_table_meta <- read_csv("data/topic_table_meta.csv", show_col_types = FALSE)
BASE_THEME <- readRDS("data/plot_theme.rds")
```

```r
sdoh_focal_topic_labels <- c(
  "Education & Community",
  "Insurance & Medicaid Coverage",
  "Policy Equity & Conditions",
  "Criminal Justice"
)

year_topic_gamma <- topic_table_meta |>
  filter(
    !is_artifact,
    !is.na(session_year),
    topic_label %in% sdoh_focal_topic_labels
  ) |>
  group_by(session_year, topic, topic_label) |>
  summarise(
    mean_gamma = mean(gamma, na.rm = TRUE),
    n_bills    = n_distinct(bill_id),
    .groups    = "drop"
  )

# Define a compact color palette for the four focal topics
FOCAL_TOPIC_COLORS <- c(
  "Education & Community"          = "#f28e2b",
  "Insurance & Medicaid Coverage"  = "#e15759",
  "Policy Equity & Conditions"     = "#b07aa1",
  "Criminal Justice"               = "#4e79a7"
)

rq4c_year_plot <- year_topic_gamma |>
  ggplot(aes(
    x     = session_year,
    y     = mean_gamma,
    color = topic_label,
    group = topic_label
  )) +
  geom_line(linewidth = 1.0) +
  geom_point(size = 3.5, alpha = 0.85) +
  scale_color_manual(values = FOCAL_TOPIC_COLORS, name = "Topic") +
  scale_x_continuous(breaks = c(2021, 2022, 2023, 2024)) +
  scale_y_continuous(labels = number_format(accuracy = 0.001)) +
  labs(
    title    = "SDOH-Focal Topic Prevalence Over Time (RQ4c)",
```

```
    subtitle = "Mean \u03b3 by session year \u00b7 point size = number of bills in that year"
    x        = "Session Year",
    y        = "Mean Document\u2013Topic Proportion (\u03b3)"
  ) +
  BASE_THEME +
  theme(legend.position = "right")

ggsave("output/figures/rq4c_year_topic_trends.png",
       rq4c_year_plot, width = 10, height = 5, dpi = 300)
rq4c_year_plot
```

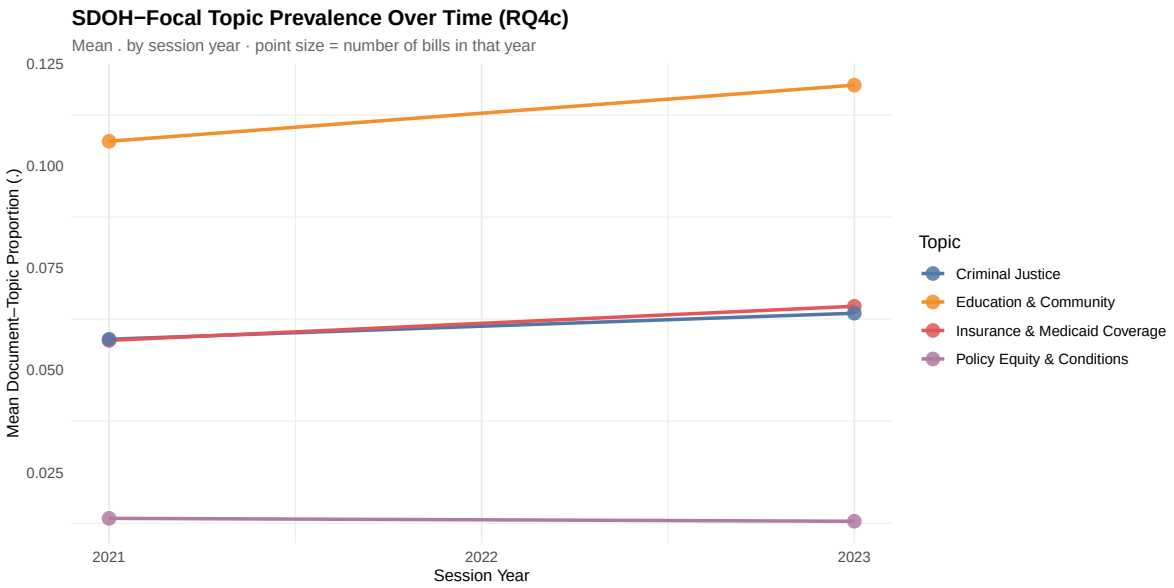

Figure 12: Figure RQ4c-ii. Mean document–topic proportion ( ) by session year for SDOH-focal topics (Education & Community; Insurance & Medicaid Coverage; Policy Equity & Conditions; Criminal Justice). Lines connect annual means across available session years. Only 2021 and 2023 appear because most states in the corpus hold biennial sessions; Florida contributes annual sessions but its bill count is too similar to other years to drive visible point-size variation.

This final RQ4c figure combines topic and domain measures by plotting, for three focal domains, each state's mean document–topic proportion for the aligned topic (blue bars) alongside that state's mean domain score (red points, rescaled), highlighting states where topic prevalence and domain scores move together.

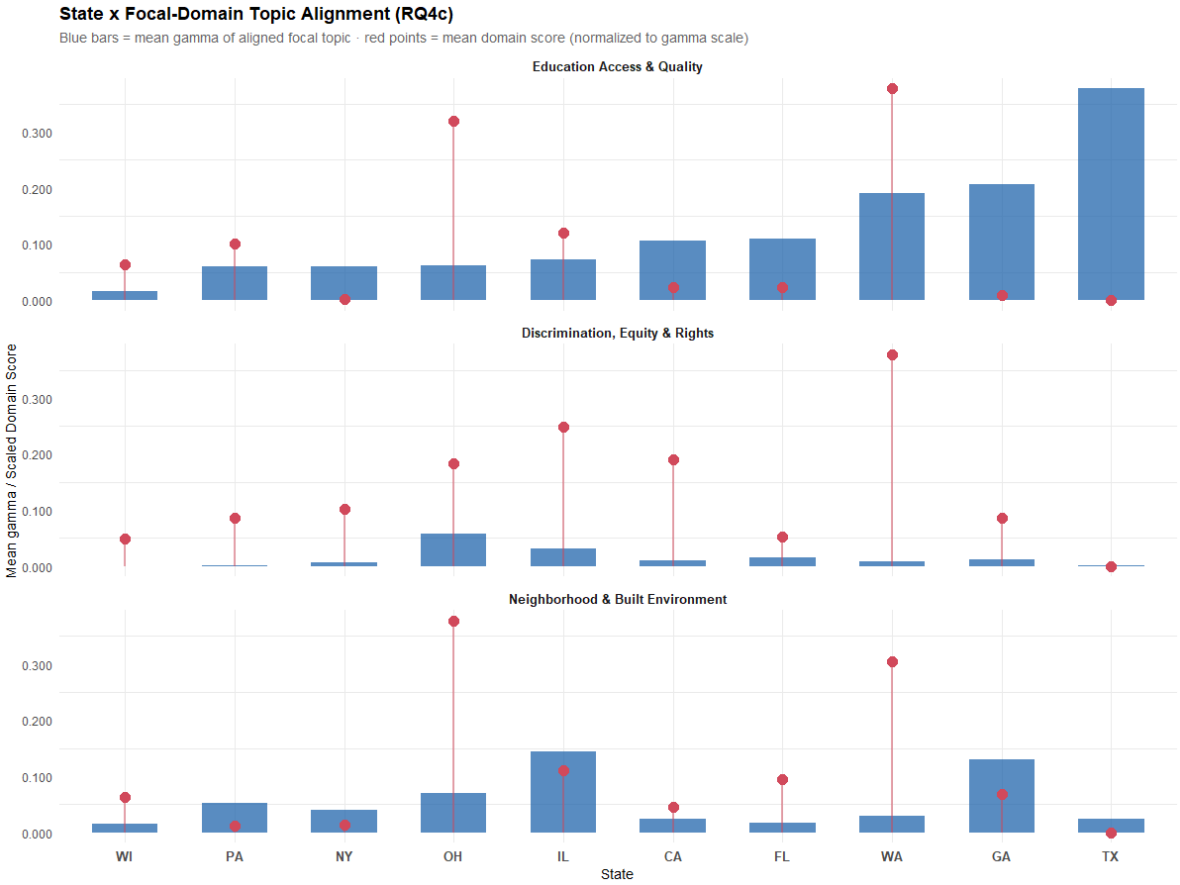

Figure 13: Figure RQ4c-iii. State variation in topic–domain alignment for three focal SDOH domains.